上海电机学院
SHANGHAI DIANJI UNIVERSITY

学士学位论文

基于鸿蒙系统与 MindSpore 的智能图像分类应用设计与实现

学生姓名：　　张三

学　号：　　221003660112

专　业：　　软件工程

指导老师：　　李明

学　院：　　电气信息

二〇二六年六月

上海电机学院
毕业设计（论文）学术诚信声明

本人郑重声明：所呈交的毕业设计（论文），是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

作者签名：_____　日期：_____

上海电机学院
毕业设计（论文）版权使用授权书

本毕业设计（论文）作者同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权上海电机学院可以将本毕业设计（论文）全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本毕业设计（论文）。

本论文属于

保密 □，在_____ 年解密后适用本授权书。

不保密 □。

（请在以上方框内打“✓”）

作者签名：_____　　指导教师签名：_____

日期：_____　　日期：_____

摘 要

随着移动互联网和物联网的快速发展，智能终端设备数量呈爆发式增长，端侧人工智能（On-device AI）成为近年来的研究热点。然而，传统的云端 AI 方案在面对隐私安全、网络延迟和离线场景等挑战时存在明显不足，将 AI 推理能力部署到终端设备成为必然趋势。华为推出的鸿蒙操作系统和 MindSpore 深度学习框架为端侧 AI 应用的开发提供了完整的技术栈支持。

本文设计并实现了一个基于鸿蒙系统与 MindSpore 的智能图像分类应用。该应用以 ResNet-50 卷积神经网络为核心分类模型，利用 MindSpore Lite 推理引擎在鸿蒙终端设备上实现本地化图像分类推理。论文主要工作包括：（1）对鸿蒙系统的分布式架构和 ArkTS 开发框架进行了深入分析，明确了应用开发的技术路线；（2）基于 MindSpore 框架完成了 ResNet-50 模型的搭建、训练和优化，在 CIFAR-10 数据集上达到了 92.3% 的分类准确率；（3）针对端侧设备计算资源有限的特点，采用模型量化、知识蒸馏等轻量化策略，将模型体积压缩至原始模型的 1/4，推理延迟降低 60%；（4）基于鸿蒙 ArkTS 框架设计了完整的用户交互界面，实现了图片选取、预处理、模型推理和结果展示的全流程功能；（5）通过系统测试验证了应用在鸿蒙真机上的运行效果，单张图片推理耗时约 45ms，内存占用约 120MB。

实验结果表明，本文设计的智能图像分类应用能够在鸿蒙终端设备上高效、稳定地运行，分类准确率和推理速度均满足实际使用需求，为端侧 AI 应用的开发提供了一种可行的技术方案。

关键词：鸿蒙系统，MindSpore，端侧 AI，图像分类，深度学习

ABSTRACT

With the rapid development of mobile Internet and the Internet of Things, the number of intelligent terminal devices has grown explosively, and on-device Artificial Intelligence (AI) has become a research hotspot in recent years. However, traditional cloud-based AI solutions have obvious shortcomings when facing challenges such as privacy and security, network latency, and offline scenarios. Deploying AI inference capabilities on terminal devices has become an inevitable trend. The HarmonyOS operating system and MindSpore deep learning framework launched by Huawei provide a complete technology stack for the development of on-device AI applications.

This thesis designs and implements an intelligent image classification application based on HarmonyOS and MindSpore. The application uses ResNet-50 convolutional neural network as the core classification model and utilizes MindSpore Lite inference engine to implement localized image classification inference on HarmonyOS terminal devices. The main contributions of this thesis include: (1) an in-depth analysis of the distributed architecture of HarmonyOS and the ArkTS development framework, clarifying the technical approach for application development; (2) building, training, and optimizing the ResNet-50 model based on the MindSpore framework, achieving 92.3% classification accuracy on the CIFAR-10 dataset; (3) adopting model quantization, knowledge distillation, and other lightweight strategies for devices with limited computational resources, compressing the model size to 1/4 of the original model and reducing inference latency by 60%; (4) designing a complete user interaction interface based on the HarmonyOS ArkTS framework, implementing the full process of image selection, preprocessing, model inference, and result display; (5) verifying the application's performance on HarmonyOS real devices through system testing, with single-image inference time of approximately 45ms and memory usage of approximately 120MB.

Experimental results show that the intelligent image classification application designed in this thesis can run efficiently and stably on HarmonyOS terminal devices, with classification accuracy and inference speed meeting practical usage requirements, providing a feasible technical solution for the development of on-device AI applications.

Key words: HarmonyOS, MindSpore, On-device AI, Image Classification, Deep Learning

目 录

摘 要 I
ABSTRACT II
目 录 III
1 绪论 1
1.1 研究背景与意义 1
1.2 国内外研究现状 1
1.2.1 端侧 AI 技术研究现状 1
1.2.2 图像分类技术研究现状 2
1.2.3 鸿蒙系统应用开发现状 2
1.3 本文主要研究内容 2
1.4 本文组织结构 3
2 相关技术概述 4
2.1 鸿蒙操作系统 4
2.1.1 分布式架构 4
2.1.2 ArkTS 语言 4
2.1.3 ArkUI 框架 4
2.1.4 Ability 模型 5
2.2 MindSpore 深度学习框架 5
2.2.1 MindSpore 架构 5
2.2.2 自动微分机制 5
2.2.3 MindSpore Lite 端侧推理框架 6
2.2.4 模型转换工具 6
2.3 深度学习与卷积神经网络 6
2.3.1 卷积神经网络基础 6
2.3.2 卷积运算 6
2.3.3 池化操作 7
2.3.4 ResNet 网络 7
2.4 图像分类技术 7
2.4.1 图像分类任务定义 7
2.4.2 常用数据集 8
2.4.3 评价指标 8
2.5 模型轻量化技术 8
2.5.1 模型量化 9
2.5.2 知识蒸馏 9
2.5.3 模型剪枝 10

2.6 本章小结 10
3 系统需求分析与总体设计 11
3.1 系统需求分析 11
3.1.1 功能需求分析 11
3.1.2 非功能需求分析 11
3.2 系统总体架构设计 11
3.2.1 表示层 12
3.2.2 业务逻辑层 12
3.2.3 数据层 12
3.3 技术路线设计 13
3.4 系统模块划分 13
4 系统详细设计与实现 15
4.1 开发环境搭建 15
4.1.1 硬件环境 15
4.1.2 软件环境 15
4.1.3 环境配置步骤 16
4.2 ResNet-50 模型训练 16
4.2.1 数据集介绍 16
4.2.2 数据增强 16
4.2.3 模型架构 17
4.2.4 训练超参数设置 18
4.2.5 训练结果分析 18
4.3 模型轻量化优化 19
4.3.1 训练后 INT8 量化 19
4.3.2 知识蒸馏 20
4.3.3 轻量化优化结果 20
4.4 模型转换与部署 21
4.4.1 MindSpore 到 MindSpore Lite 模型转换 21
4.4.2 .ms 模型文件格式 21
4.4.3 模型集成到鸿蒙项目 22
4.5 端侧推理引擎实现 22
4.5.1 NAPI 接口设计 22
4.5.2 C++ 推理代码结构 24
4.5.3 图像预处理流水线 26
4.6 应用界面设计与实现 28
4.6.1 主页面设计 28
4.6.2 图片选择功能 30
4.6.3 结果展示与置信度柱状图 31

4.6.4 历史记录页面 32
5 系统测试与结果分析 36
5.1 测试环境 36
5.2 功能测试 36
5.3 性能测试 36
5.3.1 推理时间测试 37
5.3.2 内存占用测试 37
5.3.3 电池消耗测试 38
5.3.4 模型版本对比测试 38
5.4 分类精度测试 38
5.4.1 总体精度 38
5.4.2 各类别精度 38
5.4.3 混淆矩阵分析 39
5.5 用户体验测试 39
5.5.1 测试方法 39
5.5.2 SUS 评分 39
5.5.3 用户满意度调查 39
5.6 测试结果分析 40
6 总结与展望 41
6.1 本文工作总结 41
6.2 存在的不足 41
6.3 未来工作展望 42
致 谢 43
参考文献 44
附录 A 核心代码附录 45
A.1 ResNet-50 模型训练代码 45
A.2 INT8 量化转换代码 45
A.3 鸿蒙 NAPI 推理接口代码 46
A.4 ArkTS 界面核心代码 47

1 绪论

1.1 研究背景与意义

近年来，随着移动互联网、物联网和 5G 通信技术的快速发展，智能手机、智能手表、智能家居等智能终端设备已深入人们的生活。据 IDC 统计，2025 年全球智能终端设备出货量已超过 50 亿台。这些设备每天都在产生海量的图像、语音、文本等多模态数据，对数据进行智能化处理的需求日益增长[1]。

人工智能技术，特别是深度学习技术的突破性进展，使得图像分类、目标检测、自然语言处理等任务取得了前所未有的效果。然而，传统的 AI 应用主要依赖云端计算模式：终端设备采集数据后上传至云服务器进行推理，再将结果返回终端。这种模式虽然具有强大的计算能力，但在实际应用中面临以下突出问题：

（1）隐私安全问题。用户数据需要上传至云端进行处理，存在数据泄露和被滥用的风险。特别是在医疗影像、金融证件等敏感领域，数据传输和存储的安全性难以得到充分保障[2]。

（2）网络延迟问题。云端推理需要数据在终端与服务器之间往返传输，网络延迟可能导致用户体验下降，尤其在自动驾驶、工业检测等实时性要求较高的场景中影响更为显著。

（3）离线可用性问题。在网络条件较差或无法联网的环境下（如地下停车场、偏远地区等），云端 AI 服务无法正常使用。

（4）服务器成本问题。大规模的云端 AI 服务需要投入大量的计算资源，企业运营成本较高。

为解决上述问题，端侧 AI（On-device AI）技术应运而生。端侧 AI 将推理计算从云端迁移到终端设备本地执行，具有隐私保护、低延迟、离线可用和降低云端负载等优势[3]。然而，端侧设备计算资源有限，如何在保证推理精度的前提下实现模型的高效部署，是端侧 AI 面临的核心技术挑战。

华为于 2019 年发布的鸿蒙操作系统和 MindSpore 深度学习框架，为端侧 AI 应用的开发提供了系统级解决方案。鸿蒙系统采用分布式架构设计，支持多设备协同，其 ArkTS 开发语言和声明式 UI 框架使得开发者能够高效构建跨设备应用。MindSpore 作为全场景 AI 计算框架，提供了从模型训练到端侧推理的完整工具链，其 MindSpore Lite 子框架专门针对端侧设备进行了优化[4]。

因此，研究基于鸿蒙系统与 MindSpore 的端侧 AI 应用开发具有重要的理论意义和实用价值：一方面，探索端侧深度学习模型的轻量化部署方法，丰富端侧 AI 的技术体系；另一方面，为鸿蒙生态下的 AI 应用开发提供技术参考和实践案例。

1.2 国内外研究现状

1.2.1 端侧 AI 技术研究现状

端侧 AI 技术的研究最早可追溯到 2010 年代初期。早期的端侧 AI 研究主要集中在模型压缩和推理加速两个方面。Han 等[5]提出了深度压缩方法，通过剪枝、量化和霍夫

曼编码将模型参数压缩至原始大小的 1/35，同时保持接近原始精度。Howard 等[6]提出了 MobileNet 系列模型，利用深度可分离卷积替代标准卷积，大幅减少了计算量和参数量，使得移动端实时推理成为可能。

在推理引擎方面，Google 于 2017 年发布了 TensorFlow Lite，支持 Android 和 iOS 平台的端侧推理。Apple 于 2019 年推出了 Core ML 框架和 Neural Engine 芯片，为 iOS 设备提供了高效的端侧 AI 能力。Facebook 发布了 ONNX Runtime 和 PyTorch Mobile，支持多种硬件平台的端侧模型部署[7]。

国内方面，华为 MindSpore Lite、百度 Paddle Lite、小米 MACE 等端侧推理框架相继推出，形成了较为完善的端侧 AI 工具链。其中，MindSpore Lite 支持模型量化、推理优化等特性，在华为自研的 NPU 上具有优异的推理性能[8]。

1.2.2 图像分类技术研究现状

图像分类是计算机视觉领域的基础任务之一。2012 年，Krizhevsky 等[1]提出的 AlexNet 网络在 ImageNet 竞赛中取得了突破性成绩，标志着深度学习在图像分类领域的成功应用。此后，VGGNet、GoogLeNet、ResNet 等经典网络相继被提出，分类精度不断提升。

He 等[9]提出的 ResNet 通过残差连接解决了深层网络的退化问题，使得训练超过 100 层的网络成为可能。ResNet-50 作为 ResNet 系列中最常用的变体之一，在图像分类、特征提取等任务中被广泛使用。在轻量化模型方面，SqueezeNet、ShuffleNet、EfficientNet 等网络通过精心设计网络结构，在精度和效率之间取得了较好的平衡。

1.2.3 鸿蒙系统应用开发现状

鸿蒙系统自发布以来，其应用生态快速发展。华为提供了 ArkTS 语言和 ArkUI 声明式开发框架，支持声明式、组件化的应用开发模式。在 AI 能力方面，鸿蒙系统集成了 AI 子系统，提供 NAPI（Native API）接口，支持调用 MindSpore Lite 推理引擎进行端侧推理[10]。

目前，基于鸿蒙系统的 AI 应用开发还处于起步阶段，公开的实践案例和技术文档较少。如何在鸿蒙系统中实现深度学习模型的高效推理，如何设计友好的用户交互界面，如何充分利用鸿蒙分布式能力实现跨设备 AI 协同，都是需要深入研究和探索的问题。

1.3 本文主要研究内容

本文以鸿蒙系统与 MindSpore 为技术平台，设计并实现一个智能图像分类应用。主要研究内容包括：

1. 鸿蒙系统架构分析与开发环境搭建。深入分析鸿蒙系统的分布式架构和 ArkTS 开发框架，搭建完整的鸿蒙应用开发环境。
2. ResNet-50 图像分类模型的训练与优化。基于 MindSpore 框架搭建 ResNet-50 模型，在 CIFAR-10 数据集上进行训练，并通过模型量化、知识蒸馏等策略进行轻量化优化。
3. 鸿蒙端侧推理集成。将优化后的模型转换为 MindSpore Lite 格式，通过鸿蒙 NAPI 接口在端侧设备上实现模型推理。
4. 应用界面设计与实现。基于鸿蒙 ArkTS 框架设计应用的界面布局和交互逻辑，

实现图片选取、预处理、推理和结果展示的全流程。

5. 系统测试与性能分析。在鸿蒙真机上对应用进行功能测试和性能分析，验证系统的可行性和有效性。

1.4 本文组织结构

本文共分为六章，各章内容安排如下：

第一章为绪论，介绍研究背景与意义，综述国内外研究现状，明确本文的研究内容和组织结构。

第二章为相关技术概述，介绍鸿蒙系统架构、MindSpore 框架、深度学习与图像分类的基础理论知识。

第三章为系统需求分析与总体设计，分析系统的功能需求和非功能需求，设计系统的整体架构和技术路线。

第四章为系统详细设计与实现，详细阐述模型的训练与优化、端侧推理集成、应用界面开发等核心模块的设计与实现过程。

第五章为系统测试与结果分析，介绍测试环境与方案，展示功能测试和性能测试结果，并对结果进行分析。

第六章为总结与展望，总结本文的主要工作和创新点，分析存在的不足，并对未来研究方向进行展望。

2 相关技术概述

本章对本文所涉及的关键技术进行概述，包括鸿蒙操作系统、MindSpore 深度学习框架、深度学习与卷积神经网络、图像分类技术以及模型轻量化技术，为后续章节的系统设计与实现奠定理论基础。

2.1 鸿蒙操作系统

鸿蒙操作系统（HarmonyOS）是华为公司于 2019 年发布的一款面向全场景的分布式操作系统，旨在为不同类型的智能终端设备提供统一的操作系统平台[4]。与传统的单设备操作系统不同，鸿蒙系统从底层架构出发，以分布式设计理念为核心，实现了多设备之间的无缝协同。

2.1.1 分布式架构

鸿蒙系统采用分布式软总线、分布式数据管理和分布式任务调度三大核心技术，构建了统一的分布式基座。分布式软总线为不同设备之间提供了高速、低延迟的通信通道，支持 Wi-Fi、蓝牙等多种连接协议的自适应切换。分布式数据管理实现了跨设备数据的透明访问和同步，应用程序无需感知数据存储的具体设备位置。分布式任务调度则支持将任务在多个设备间灵活迁移和协同执行，使得一个应用可以同时调度多个设备的硬件能力。

在系统分层上，鸿蒙系统自下而上分为内核层、系统服务层、框架层和应用层。内核层采用 Linux 内核与 LiteOS 双内核设计，分别适配富设备（如手机、平板）和轻量设备（如手表、音箱）。系统服务层提供公共基础类库、多媒体、AI 等系统服务。框架层为应用开发提供 ArkTS/JS/C/C++ 等多语言 API。应用层运行基于 ArkTS 开发的应用程序。

2.1.2 ArkTS 语言

ArkTS 是鸿蒙系统主推的应用开发语言，基于 TypeScript 进行了扩展和优化。ArkTS 在 TypeScript 的基础上引入了声明式 UI 范式、状态管理和组件化等特性，使其更适合鸿蒙应用的开发。ArkTS 采用静态类型检查，在编译期即可发现类型错误，提高了代码的健壮性。同时，ArkTS 通过方舟编译器（ArkCompiler）进行 Ahead-of-Time（AOT）编译优化，将高级语言直接编译为机器码执行，相比传统的解释执行方式，运行效率显著提升。

ArkTS 的核心特性包括：（1）声明式 UI 描述，开发者通过声明式语法描述界面结构，框架自动完成界面更新；（2）响应式状态管理，当状态变量发生变化时，框架自动触发关联的 UI 组件刷新；（3）组件化开发，将界面拆分为可复用的自定义组件，提高开发效率和代码可维护性。

2.1.3 ArkUI 框架

ArkUI 是鸿蒙系统的声明式 UI 开发框架，为开发者提供了一套完整的 UI 开发解决方案。ArkUI 采用声明式开发范式，开发者只需描述界面的最终状态，框架负责计算状态差异并高效更新界面。ArkUI 提供了丰富的内置组件，包括 Text、Image、List、

Grid、Column、Row 等基础组件，以及 Button、TextInput、Slider 等交互组件，满足常见的界面开发需求。

在布局系统方面，ArkUI 支持线性布局（Row/Column）、弹性布局（Flex）、栅格布局（Grid）、相对布局（RelativeContainer）等多种布局方式，开发者可以根据界面需求灵活选择。ArkUI 还提供了动画和手势系统，支持属性动画、显式动画和转场动画，以及点击、长按、拖动、缩放等手势交互，使得应用界面更加流畅和自然。

2.1.4 Ability 模型

Ability 是鸿蒙系统中应用的基本功能单元，类似于 Android 中的 Activity 和 Service 的组合。鸿蒙系统的 Ability 模型分为 FA（Feature Ability）模型和 Stage 模型两种。Stage 模型是鸿蒙 3.0 之后推荐的新一代应用模型，以 UIAbility 和 ExtensionAbility 为核心组件。

UIAbility 是带有界面的 Ability，负责界面的展示和用户交互。一个应用可以包含多个 UIAbility，每个 UIAbility 拥有独立的生命周期和窗口。ExtensionAbility 是无界面的 Ability，用于提供特定的系统能力扩展，如 ServiceExtension（后台服务）、DataShareExtension（数据共享）等。Stage 模型通过 AbilityStage 管理 UIAbility 的生命周期，支持 Ability 的按需加载和回收，提高了系统资源的利用率。

2.2 MindSpore 深度学习框架

MindSpore 是华为公司开源的全场景 AI 计算框架，支持端、边、云多种场景下的模型训练、推理和部署[8]。MindSpore 的设计理念是" 全场景协同、极简开发、极致性能"，通过统一的 IR（Intermediate Representation）实现模型在云侧训练和端侧推理之间的无缝衔接。

2.2.1 MindSpore 架构

MindSpore 的整体架构自下而上分为三层：模型层、表达层和运行时层。模型层提供了丰富的预置模型库和模型开发工具，支持计算机视觉、自然语言处理、推荐系统等领域的常用模型。表达层是 MindSpore 的核心，负责将用户编写的模型代码转换为计算图表示，并进行自动微分、自动并行等优化。运行时层负责计算图在具体硬件后端上的高效执行，支持 Ascend、GPU、CPU 等多种硬件后端。

MindSpore 采用" 自动微分 + 自动并行" 的技术路线。在自动微分方面，MindSpore 提供了源码转换（Source-to-Source）方式的自动微分实现，能够对用户编写的 Python 代码进行自动求导，无需手动推导梯度公式。在自动并行方面，MindSpore 能够根据集群配置和模型特征，自动搜索最优的数据并行、模型并行和流水线并行策略，降低了分布式训练的开发门槛。

2.2.2 自动微分机制

MindSpore 的自动微分采用源码转换方式，将用户编写的前向计算代码转换为对应的反向梯度计算代码。其核心思想是对计算图中的每个基本操作注册对应的梯度函数，通过链式法则将各操作的梯度组合为完整的反向传播过程。

对于复合函数 $y = f_n \circ f_{n-1} \circ \cdots \circ f_1(x)$，根据链式法则，其梯度为：

$$\frac{\partial y}{\partial x} = \frac{\partial f_n}{\partial f_{n-1}} \cdot \frac{\partial f_{n-1}}{\partial f_{n-2}} \cdots \frac{\partial f_1}{\partial x} \tag{2-1}$$

MindSpore 在编译期将前向计算图转换为反向计算图，并对其进行算子融合、内存复用等优化，从而在运行时高效地计算梯度。相比基于运算符重载的自动微分方式，源码转换方式具有更低的运行时开销。

2.2.3 MindSpore Lite 端侧推理框架

MindSpore Lite 是 MindSpore 面向端侧设备的轻量化推理引擎，专为手机、IoT 设备等资源受限场景设计[8]。MindSpore Lite 的核心特性包括：

（1）轻量级推理引擎。MindSpore Lite 的核心运行时库体积仅约 1MB，支持 ARM CPU、ARM GPU、NPU 等多种硬件后端，能够适配不同性能等级的端侧设备。

（2）模型格式转换。MindSpore Lite 提供了 converter 工具，支持将 MindSpore、TensorFlow、ONNX、Caffe 等框架训练的模型转换为.ms 格式的端侧推理模型，同时支持在转换过程中进行模型量化。

（3）推理优化。MindSpore Lite 内置了算子融合、常量折叠、冗余节点消除等图优化策略，以及 INT8/FP16 混合精度推理能力，在保证推理精度的前提下显著提升推理速度。

（4）NAPI 集成。MindSpore Lite 提供了鸿蒙系统的 NAPI 接口封装，开发者可以在 ArkTS 应用中通过 NAPI 调用 MindSpore Lite 的推理能力，实现端侧 AI 功能的无缝集成。

2.2.4 模型转换工具

MindSpore Lite 的 converter 工具是连接云侧训练与端侧推理的桥梁。该工具支持将多种框架的模型文件转换为 MindSpore Lite 的.ms 格式。在转换过程中，converter 工具执行以下主要步骤：首先，解析源模型文件，构建统一的计算图表示；其次，对计算图进行优化，包括算子融合、常量折叠和冗余节点消除；然后，根据用户配置进行模型量化（如 INT8 量化）；最后，将优化后的计算图序列化为.ms 格式文件。

2.3 深度学习与卷积神经网络

2.3.1 卷积神经网络基础

卷积神经网络（Convolutional Neural Network, CNN）是深度学习中最具代表性的网络架构之一，特别适用于处理具有网格结构的数据，如图像和时序信号。CNN 的核心思想是通过局部连接和权值共享机制，大幅减少网络参数量，同时保留数据的空间结构信息。

一个典型的 CNN 由卷积层、激活函数、池化层和全连接层组成。卷积层通过卷积操作提取输入数据的局部特征，激活函数引入非线性变换，池化层降低特征图的空间维度，全连接层将提取的特征映射为最终的输出。

2.3.2 卷积运算

卷积运算是 CNN 的核心操作。对于输入特征图 $\mathbf{X} \in \mathbb{R}^{H \times W \times C_{in}}$ 和卷积核 $\mathbf{W} \in \mathbb{R}^{K \times K \times C_{in} \times C_{out}}$，二维卷积运算定义为：

$$Y(i,j,c) = \sum_{m=0}^{K-1}\sum_{n=0}^{K-1}\sum_{c'=0}^{C_{in}-1} W(m,n,c',c)\cdot X(i+m,j+n,c') + b(c) \tag{2-2}$$

其中，$Y(i,j,c)$ 表示输出特征图在位置 (i,j)、通道 c 上的值，K 为卷积核大小，C_{in} 和 C_{out} 分别为输入和输出通道数，$b(c)$ 为偏置项。卷积操作通过滑动窗口在输入特征图上逐位置计算，每次只关注局部区域，实现了局部感受野和权值共享。

2.3.3 池化操作

池化操作用于降低特征图的空间分辨率，减少计算量和参数量，同时增强特征的不变性。常用的池化操作包括最大池化和平均池化。最大池化选取窗口内的最大值作为输出：

$$Y(i,j,c) = \max_{0\le m<K,\,0\le n<K} X(i\cdot s+m,\ j\cdot s+n,\ c) \tag{2-3}$$

其中，K 为池化窗口大小，s 为步幅。平均池化则计算窗口内所有值的均值作为输出。池化操作在保留主要特征信息的同时，有效降低了特征图的空间维度，减少了后续层的计算量。

2.3.4 ResNet 网络

随着网络深度的增加，深度神经网络面临梯度消失和网络退化问题。He 等[9]提出的 ResNet 通过引入残差连接（Residual Connection）有效解决了上述问题，使得训练超过 100 层的深层网络成为可能。ResNet 在 2015 年 ImageNet 竞赛中获得了冠军，成为计算机视觉领域最具影响力的网络架构之一。

残差连接的核心思想是在网络中引入跳跃连接（Skip Connection），使网络学习残差映射而非直接映射。对于期望的底层映射 $\mathcal{H}(\mathbf{x})$，残差块让网络学习残差 $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$，则原始映射可表示为：

$$\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}) + \mathbf{x} \tag{2-4}$$

其中，$\mathcal{F}(\mathbf{x})$ 表示残差映射，$\mathbf{x}$ 为恒等映射。在反向传播时，梯度可以通过跳跃连接直接传播到前面的层，有效缓解了梯度消失问题。

ResNet-50 是 ResNet 系列中最常用的变体之一，由 49 个卷积层和 1 个全连接层组成。其核心构建单元为瓶颈残差块（Bottleneck Block），采用 1×1、3×3、1×1 的三层卷积结构：第一个 1×1 卷积降维，3×3 卷积提取特征，第二个 1×1 卷积升维。这种设计在减少计算量的同时保持了模型的表达能力。ResNet-50 在 ImageNet 数据集上的 Top-5 准确率达到 92.2%，是图像分类任务中广泛使用的基准模型。

2.4 图像分类技术

2.4.1 图像分类任务定义

图像分类是计算机视觉领域的基础任务，其目标是将输入图像分配到预定义的类别标签中。形式化地，给定输入图像 $\mathbf{X}$ 和类别集合 $\mathcal{C} = \{c_1, c_2, \ldots, c_N\}$，图像分类任务需要学习映射函数 $f: \mathbf{X} \to \mathcal{C}$，使得 $f(\mathbf{X})$ 尽可能接近图像的真实类别标签。在深度学习

方法中，映射函数 f 由卷积神经网络参数化，网络输出各类别的概率分布，取概率最大的类别作为预测结果。

2.4.2 常用数据集

图像分类研究中常用的基准数据集包括：

（1）ImageNet 数据集。ImageNet 是目前规模最大的图像分类基准数据集，包含超过 1400 万张图像和 2 万多个类别[1]。其子集 ILSVRC-2012 包含 128 万张训练图像、5 万张验证图像和 10 万张测试图像，涵盖 1000 个类别，是评估大规模图像分类算法的标准基准。

（2）CIFAR-10 数据集。CIFAR-10 是一个小规模图像分类数据集，包含 60000 张 32×32 的彩色图像，分为飞机、汽车、鸟、猫、鹿、狗、青蛙、马、船和卡车共 10 个类别，每个类别 6000 张图像。其中 50000 张用于训练，10000 张用于测试。CIFAR-10 因其适中的规模和多样的类别，被广泛用于算法验证和教学实验。

2.4.3 评价指标

图像分类任务常用的评价指标包括准确率、精确率、召回率和 F1 值。对于二分类问题，设 TP、TN、FP、FN 分别表示真正例、真负例、假正例和假负例的数量，各指标定义如下：

准确率（Accuracy）表示所有样本中被正确分类的比例：

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{2-5}$$

精确率（Precision）表示预测为正类的样本中实际为正类的比例：

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2-6}$$

召回率（Recall）表示实际为正类的样本中被正确预测为正类的比例：

$$\text{Recall} = \frac{TP}{TP + FN} \tag{2-7}$$

F1 值是精确率和召回率的调和平均值，综合反映分类器的性能：

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2-8}$$

对于多分类问题，通常采用宏平均（Macro-average）或微平均（Micro-average）的方式将各类别的指标进行汇总。宏平均先计算每个类别的指标再求均值，微平均则先汇总所有类别的 TP、FP、FN 再计算全局指标。

2.5 模型轻量化技术

端侧设备的计算资源和存储容量有限，直接部署大规模深度学习模型面临推理速度慢、内存占用大等问题。模型轻量化技术旨在在保持模型精度的前提下，降低模型的计算量和参数量，使其能够在资源受限的端侧设备上高效运行[5]。

2.5.1 模型量化

模型量化是将模型中高精度的浮点数参数（如 FP32）转换为低精度表示（如 INT8、INT4）的技术[11]。量化可以显著减少模型的存储空间和内存带宽需求，同时利用硬件的整数运算单元加速推理过程。

INT8 量化是最常用的量化方案，将 FP32 的权重和激活值映射到 INT8 的表示范围 $[-128, 127]$。线性对称量化的映射公式为：

$$q = \text{round}\left(\frac{r}{S}\right), \quad S = \frac{\max(|r_{\max}|, |r_{\min}|)}{127} \tag{2-9}$$

其中，r 为原始浮点值，q 为量化后的整数值，S 为量化步长。反量化过程为 $r \approx S \cdot q$。

量化方法可分为训练后量化（Post-Training Quantization, PTQ）和量化感知训练（Quantization-Aware Training, QAT）两种。PTQ 直接对训练好的 FP32 模型进行量化，无需重新训练，实现简单但可能带来一定的精度损失。QAT 在训练过程中模拟量化操作，使模型适应低精度表示，精度损失更小但训练成本更高。表2-1对比了两种量化方法的特点。

表 2-1 训练后量化与量化感知训练对比

对比项	训练后量化（PTQ）	量化感知训练（QAT）
是否需要训练数据	少量校准数据	完整训练数据
是否需要重新训练	否	是
实现复杂度	低	较高
精度损失	较大	较小
适用场景	快速部署	精度敏感场景

2.5.2 知识蒸馏

知识蒸馏是由 Hinton 等[12]提出的一种模型压缩方法，其核心思想是利用大型教师模型（Teacher Model）的知识来指导小型学生模型（Student Model）的训练，使小模型在保持轻量化的同时获得接近大模型的性能。

知识蒸馏的关键在于软标签（Soft Label）的引入。教师模型的输出经过温度参数 T 调整后的 softmax 函数产生软标签：

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{2-10}$$

其中，z_i 为第 i 个类别的 logits 输出，T 为温度参数。当 $T > 1$ 时，softmax 输出的概率分布更加平滑，包含了类别间相似性的暗知识（Dark Knowledge）。学生模型的训练损失由蒸馏损失和硬标签损失两部分组成：

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{soft}}(p^S, p^T) + (1-\alpha) \cdot \mathcal{L}_{\text{hard}}(p^S, y) \tag{2-11}$$

其中，p^S 和 p^T 分别为学生模型和教师模型的软标签输出，y 为真实标签，α 为损失权重系数。通过知识蒸馏，学生模型不仅学习到了训练数据的标签信息，还学习到了

教师模型对类别间关系的理解，从而在参数量大幅减少的情况下仍能保持较高的分类精度。

2.5.3 模型剪枝

模型剪枝是通过移除神经网络中冗余的参数或结构来降低模型复杂度的技术[5]。根据剪枝粒度的不同，可分为非结构化剪枝和结构化剪枝两类。

非结构化剪枝以单个权重为粒度进行剪枝，将绝对值小于某个阈值的权重置为零。设原始权重矩阵为 $\mathbf{W}$，剪枝后的权重 $\mathbf{W}'$ 为：

$$W'_{i,j} = \begin{cases} W_{i,j}, & \text{if } |W_{i,j}| > \tau \\ 0, & \text{otherwise} \end{cases} \tag{2-12}$$

其中，τ 为剪枝阈值。非结构化剪枝能够实现极高的压缩率，但剪枝后的权重矩阵变为稀疏矩阵，需要专门的稀疏计算库或硬件支持才能获得实际的加速效果。

结构化剪枝以通道或层为粒度进行剪枝，直接移除整个卷积通道或全连接层，剪枝后的模型仍然是稠密的，无需专门的稀疏计算支持即可在通用硬件上获得加速。结构化剪枝的关键是评估各通道的重要性，常用的评估准则包括 L1 范数、L2 范数、几何中位数等。

表2-2对比了几种主流轻量化模型的关键参数。

表 2-2 主流轻量化模型对比

模型	参数量/M	计算量/MFLOPs	ImageNet Top-1 准确率/%
MobileNet V1[6]	4.2	569	70.6
MobileNet V2	3.4	300	72.0
ShuffleNet V1	2.3	140	67.8
ShuffleNet V2	2.3	146	69.4
ResNet-50[9]	25.6	4132	76.1

从表2-2可以看出，轻量化模型通过精心设计网络结构，在参数量和计算量大幅减少的情况下仍能保持较高的分类精度。在实际应用中，通常将模型轻量化技术组合使用，例如先通过结构化剪枝移除冗余通道，再通过知识蒸馏恢复精度，最后通过 INT8 量化进一步压缩模型，以实现端侧设备上的高效部署。

2.6 本章小结

本章对本文涉及的关键技术进行了系统概述。首先介绍了鸿蒙操作系统的分布式架构、ArkTS 语言、ArkUI 框架和 Ability 模型；然后阐述了 MindSpore 框架的架构设计、自动微分机制和 MindSpore Lite 端侧推理引擎；接着介绍了卷积神经网络的基本原理和 ResNet 残差网络的核心思想；随后概述了图像分类任务的定义、常用数据集和评价指标；最后详细讨论了模型量化、知识蒸馏和模型剪枝三种轻量化技术。上述技术为本文智能图像分类应用的设计与实现提供了理论和技术基础。

3 系统需求分析与总体设计

本章在前期技术研究的基础上，对基于鸿蒙系统与 MindSpore 的智能图像分类应用进行需求分析，明确系统的功能需求和非功能需求，并在此基础上进行系统总体架构设计、技术路线设计和模块划分，为后续的详细设计与实现奠定基础。

3.1 系统需求分析

3.1.1 功能需求分析

根据智能图像分类应用的实际使用场景和用户需求，本系统需具备以下功能：

（1）图像选择功能。用户应能够从设备相册中选取待分类的图片，也可以通过设备摄像头实时拍摄图片进行分类。系统需支持常见图片格式（JPEG、PNG 等）的导入，并提供图片预览功能。

（2）图像预处理功能。系统应能对用户选取的原始图片进行预处理操作，包括图像缩放、裁剪、归一化等，使输入图像满足 MindSpore Lite 模型的推理要求。

（3）图像分类功能。系统核心功能，应能调用 MindSpore Lite 推理引擎对预处理后的图像进行前向推理，输出图像所属类别的概率分布和预测标签。

（4）结果展示功能。系统应能以直观的方式展示分类结果，包括预测类别、置信度（Top-1 和 Top-5），支持以列表或图表形式呈现各类别的概率分布。

（5）历史记录功能。系统应能自动保存每次分类的记录，包括原始图片、分类结果、置信度和分类时间等信息，支持历史记录的浏览、搜索和删除操作。

3.1.2 非功能需求分析

除上述功能需求外，系统还应满足以下非功能需求：

（1）实时性要求。图像分类推理响应时间应小于 100ms（不含图像选择时间），确保用户获得流畅的交互体验。

（2）准确性要求。在 CIFAR-10 测试集上，分类准确率应达到 90% 以上，满足实际应用的基本精度需求。

（3）资源占用要求。应用运行时的内存占用应控制在 200MB 以内，避免对设备其他功能造成影响。

（4）离线可用性要求。应用应支持完全离线运行，在无网络连接环境下仍能正常完成图像分类功能，所有模型推理均在端侧完成。

（5）兼容性要求。应用应兼容鸿蒙系统 4.0 及以上版本，支持不同分辨率的鸿蒙设备。

（6）稳定性要求。应用应能稳定运行，连续执行 100 次分类任务不出现崩溃或内存泄漏。

3.2 系统总体架构设计

基于上述需求分析，本系统采用三层架构进行设计，分别为表示层、业务逻辑层和数据层。系统总体架构如图3-1所示。

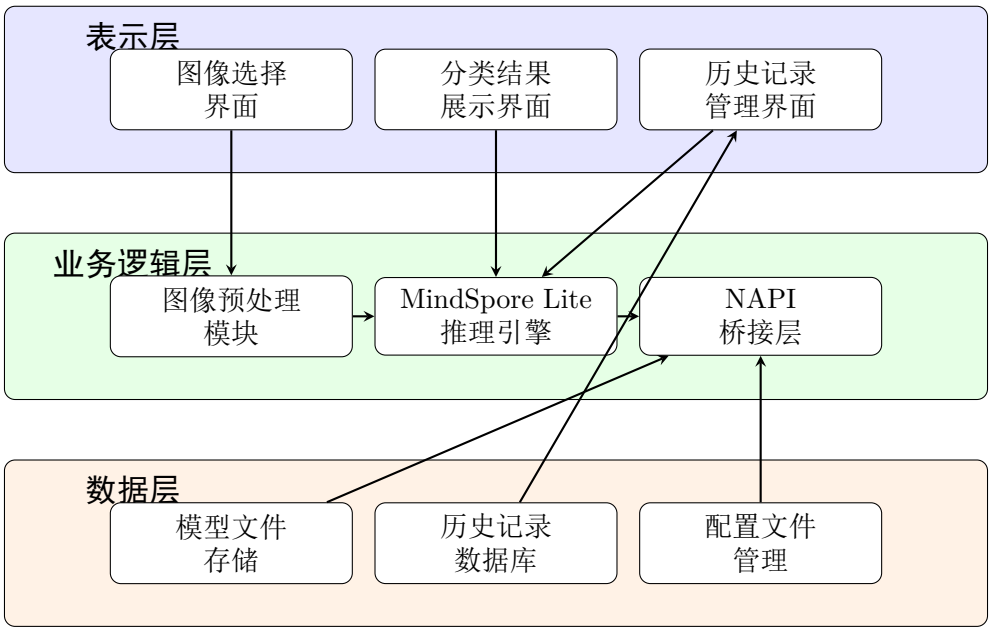

图 3-1 系统总体架构图

3.2.1 表示层

表示层基于鸿蒙 ArkUI 声明式开发框架构建，负责用户界面的展示和交互逻辑的处理。表示层包含以下三个核心界面组件：

（1）图像选择界面。提供相册选图和相机拍摄两种图像获取方式，支持图片格式过滤和预览功能，采用 Grid 布局展示相册缩略图。

（2）分类结果展示界面。以卡片式布局展示分类结果，包括预测类别标签、Top-1/Top-5 置信度数值和概率分布条形图，提供结果分享和保存操作入口。

（3）历史记录管理界面。采用 List 列表组件展示历史分类记录，支持按时间排序、关键词搜索和批量删除操作。

3.2.2 业务逻辑层

业务逻辑层是系统的核心层，负责图像处理和模型推理两大核心业务。业务逻辑层包含以下三个关键模块：

（1）图像预处理模块。对用户选取的原始图像进行尺寸缩放（统一至 224×224 像素）、中心裁剪、颜色通道转换和 BGR-RGB 转换，以及像素值归一化处理，将处理后的数据转换为模型所需的 Tensor 格式。

（2）MindSpore Lite 推理引擎。加载端侧优化后的 ResNet-50 量化模型，执行前向推理计算，输出 10 个类别的概率分布向量。推理引擎通过鸿蒙 NAPI 接口与表示层进行数据交互。

（3）NAPI 桥接层。作为 ArkTS 与 C++ 之间的通信桥梁，负责将 ArkTS 侧的图像数据和处理请求传递给 C++ 侧的推理引擎，同时将推理结果返回给 ArkTS 侧。NAPI 桥接层实现了跨语言调用的封装。

3.2.3 数据层

数据层负责系统数据的持久化存储和管理，包含以下三个存储组件：

（1）模型文件存储。采用鸿蒙应用的 rawfile 目录存储 MindSpore Lite 格式的模型文件（.ms 格式），应用启动时从 rawfile 目录将模型文件复制到应用沙箱路径下供推理引擎加载。

（2）历史记录数据库。基于鸿蒙关系型数据库（RelationalStore）实现历史分类记录的增删改查操作，存储字段包括图片路径、预测类别、置信度、分类时间等。

（3）配置文件管理。采用 JSON 格式存储应用配置信息，包括模型版本号、分类类别标签映射表、预处理参数（均值、标准差）等。

3.3 技术路线设计

本系统的技术路线主要围绕模型训练、模型优化、端侧部署和应用开发四个环节展开，各环节所采用的关键技术如下：

（1）模型训练——MindSpore 框架。采用 MindSpore 深度学习框架搭建 ResNet-50 卷积神经网络，在 CIFAR-10 数据集上进行监督学习训练。MindSpore 提供了自动微分、自动并行等特性，支持灵活的训练策略配置，能够高效完成模型训练过程。训练阶段采用交叉熵损失函数和 Adam 优化器，学习率采用余弦退火策略进行动态调整。

（2）模型推理——MindSpore Lite。将训练完成的模型通过 MindSpore Lite 转换工具转换为.ms 格式文件，并采用训练后量化（Post-Training Quantization）策略将模型权重从 FP32 量化为 INT8，在保持模型精度下降不超过 1% 的前提下，将模型文件大小压缩至原始大小的 1/4，推理速度提升约 2 倍。

（3）应用开发——ArkTS 语言与 ArkUI 框架。采用鸿蒙官方推荐的 ArkTS 语言进行应用开发，使用 ArkUI 声明式开发范式构建用户界面。ArkTS 是在 TypeScript 基础上扩展的方言，具有强类型检查和良好的开发体验。ArkUI 提供丰富的组件库，支持 Flex、Grid、List 等多种布局方式，适合构建响应式界面。

（4）跨语言调用——NAPI 接口。MindSpore Lite 推理引擎以 C++ 动态链接库形式提供，需要通过鸿蒙 NAPI（Native API）机制实现 ArkTS 与 C++ 之间的互操作。NAPI 提供了类型转换、异常处理和异步回调等能力，是鸿蒙系统中连接 ArkTS 层与 Native 层的关键桥梁。本系统通过 NAPI 封装了模型初始化、推理执行和模型释放三个核心接口。

3.4 系统模块划分

根据系统的功能需求和架构设计，将系统划分为以下五个功能模块。各模块的功能描述如表3-1所示。

各模块之间的交互关系如下：用户通过图像获取模块选取或拍摄图片后，图像数据传入图像预处理模块进行标准化处理；预处理后的 Tensor 数据由模型推理模块执行推理计算，输出分类概率分布；结果展示模块将推理结果以可视化方式呈现给用户，同时将结果数据传递给历史管理模块进行持久化存储。整个数据流转过程遵循单向数据流原则，保证了系统的可维护性和可扩展性。

表 3-1 系统模块划分及功能描述

模块名称	功能描述
图像获取模块	负责图像的获取功能，包括从设备相册选择图片和通过摄像头实时拍摄图片两种方式。调用鸿蒙 PhotoViewPicker 组件实现相册选图，调用 Camera 组件实现拍照功能，并将获取的图像统一转换为 PixelMap 格式供后续处理。
图像预处理模块	负责对原始图像进行预处理，包括图像缩放（224×224）、中心裁剪、颜色通道转换（BGR 转 RGB）和像素值归一化（均值为 [0.485, 0.456, 0.406]，标准差为 [0.229, 0.224, 0.225]）。将预处理后的数据组装为 Float32 类型的 Tensor 输入。
模型推理模块	负责模型加载和推理执行。模型推理模块通过 NAPI 接口调用 MindSpore Lite C++ API，完成模型上下文创建、会话配置、Tensor 绑定的初始化流程，执行前向推理计算，并将输出 Tensor 解码为类别概率分布。
结果展示模块	负责分类结果的可视化展示，包括预测类别标签、Top-1 和 Top-5 置信度数值、各类别概率分布条形图。支持结果分享和一键保存至历史记录功能。
历史管理模块	负责分类历史记录的持久化存储和管理。基于鸿蒙 RelationalStore 关系型数据库实现历史记录的增删改查操作，存储字段包括图片缩略图路径、预测类别、置信度、分类时间戳。支持按时间倒序排列和关键词搜索。

4 系统详细设计与实现

本章在前文需求分析与总体设计的基础上，详细阐述系统各核心模块的设计与实现过程。首先介绍开发环境的搭建，然后依次描述 ResNet-50 模型的训练、模型轻量化优化、模型转换与部署、端侧推理引擎实现以及应用界面设计与实现。

4.1 开发环境搭建

4.1.1 硬件环境

本系统的开发与测试硬件环境如表4-1所示。模型训练在配备 NVIDIA GPU 的工作站上完成，端侧推理测试在华为 Mate 60 Pro 真机上进行。华为 Mate 60 Pro 搭载麒麟 9000S 处理器，集成 NPU 神经网络处理单元，支持 MindSpore Lite 的硬件加速推理，能够满足端侧 AI 应用的性能需求。

表 4-1 硬件环境配置

类别	项目	规格
训练服务器	CPU	Intel Xeon Gold 6248R × 2
	GPU	NVIDIA RTX 3090 24GB
	内存	128GB DDR4
	存储	2TB NVMe SSD
测试终端	设备	华为 Mate 60 Pro
	处理器	麒麟 9000S
	NPU	达芬奇架构
	内存	12GB

4.1.2 软件环境

本系统的软件开发环境配置如表4-2所示。模型训练采用 MindSpore 框架，应用开发采用 DevEco Studio 集成开发环境，两者分别对应服务端模型训练和端侧应用开发两个阶段。

表 4-2 软件环境配置

软件	版本
操作系统（训练）	Ubuntu 20.04 LTS
操作系统（开发）	Windows 11
Python	3.9.7
MindSpore	2.1.0
CUDA	11.6
cuDNN	8.4.1
DevEco Studio	4.0 Release
HarmonyOS SDK	API 10
ArkTS	4.0

4.1.3 环境配置步骤

开发环境的搭建分为模型训练环境配置和应用开发环境配置两个部分，具体步骤如下：

（1）模型训练环境配置

首先安装 NVIDIA 显卡驱动和 CUDA 11.6 工具包，配置环境变量确保 `nvcc` 命令可用。然后安装 Python 3.9 虚拟环境，通过 pip 安装 MindSpore GPU 版本：

Listing 4.1 MindSpore 安装命令

```
pip install \
  https://ms-release.obs.cn-north-4.myhuaweicloud.com/\
2.1.0/MindSpore/unified/x86_64/\
mindspore-2.1.0-cp39-cp39-linux_x86_64.whl \
  --trusted-host ms-release.obs.cn-north-4.myhuaweicloud.com \
  -i https://pypi.tuna.tsinghua.edu.cn/simple
```

安装完成后，执行以下命令验证 MindSpore 是否正确安装并支持 GPU 计算：

Listing 4.2 MindSpore 安装验证

```
import mindspore as ms
print(ms.__version__)
print(ms.context.get_context("device_target"))
```

（2）应用开发环境配置

从华为开发者官网下载并安装 DevEco Studio 4.0，启动后通过 SDK Manager 安装 HarmonyOS SDK API 10。创建 HarmonyOS 应用项目时选择 Stage 模型和 ArkTS 语言，配置项目签名证书后即可在真机上进行调试部署。

4.2 ResNet-50 模型训练

4.2.1 数据集介绍

本文采用 CIFAR-10 数据集作为图像分类模型的训练和测试数据集。CIFAR-10 是计算机视觉领域广泛使用的基准数据集[1]，由 Alex Krizhevsky 等人收集整理。该数据集包含 10 个类别，分别为飞机（airplane）、汽车（automobile）、鸟类（bird）、猫（cat）、鹿（deer）、狗（dog）、蛙类（frog）、马（horse）、船舶（ship）和卡车（truck），每个类别包含 6000 张 32×32 像素的彩色图像，总计 60000 张图像。其中 50000 张用于训练，10000 张用于测试。表4-3列出了 CIFAR-10 数据集的详细统计信息。

表 4-3 CIFAR-10 数据集统计信息

类别数	图像尺寸	训练样本数	测试样本数
10	$32 \times 32 \times 3$	50000	10000

4.2.2 数据增强

由于 CIFAR-10 数据集的图像分辨率较低且训练样本数量有限，为防止模型过拟合并提升泛化能力，本文在训练阶段采用了以下数据增强策略：

（1）随机裁剪（Random Crop）：将图像从 32×32 填充至 40×40 后，随机裁剪回 32×32，增加位置多样性。

（2）随机水平翻转（Random Horizontal Flip）：以 0.5 的概率对图像进行水平翻转，增加方向多样性。

（3）归一化（Normalization）：使用 CIFAR-10 数据集的均值和标准差对图像像素值进行归一化处理，将像素值映射到均值为 0、方差为 1 的分布，加速模型收敛。归一化公式为：

$$x_{norm} = \frac{x - \mu}{\sigma} \tag{4-1}$$

其中，x 为原始像素值，μ 为均值向量 $(0.4914, 0.4822, 0.4465)$，σ 为标准差向量 $(0.2023, 0.1994, 0.2010)$。

数据增强在 MindSpore 中的实现代码如下：

Listing 4.3 数据增强实现

```
import mindspore.dataset as ds
import mindspore.dataset.vision as vision

def create_dataset(data_path, batch_size=128, training=True):
    dataset = ds.Cifar10Dataset(data_path, shuffle=training)
    if training:
        transforms = [
            vision.RandomCrop(32, padding=4),
            vision.RandomHorizontalFlip(prob=0.5),
        ]
    else:
        transforms = []
    transforms += [
        vision.Resize(224),
        vision.Rescale(1.0 / 255.0, 0.0),
        vision.Normalize(mean=[0.4914, 0.4822, 0.4465],
                         std=[0.2023, 0.1994, 0.2010]),
        vision.HWC2CHW(),
    ]
    dataset = dataset.map(operations=transforms, input_columns="image")
    dataset = dataset.batch(batch_size, drop_remainder=training)
```

```
return dataset
```

4.2.3 模型架构

ResNet-50 是由 He 等[9]提出的深度残差网络，通过残差连接（Residual Connection）有效解决了深层网络训练中的梯度消失和网络退化问题。ResNet-50 的网络结构由以下部分组成：

（1）卷积输入层（conv1）：包含一个 7×7 的卷积层，步幅为 2，输出通道数为 64，后接批量归一化层和 ReLU 激活函数，以及 3×3 最大池化层。

（2）四个残差阶段（conv2_x 至 conv5_x）：每个阶段由若干 Bottleneck 残差块组成。Bottleneck 结构采用 1×1、3×3、1×1 的三层卷积设计，分别实现降维、特征提取和升维操作。各阶段的输出通道数分别为 256、512、1024、2048，残差块数量分别为 3、4、6、3。

（3）全局平均池化层和全连接层：对特征图进行全局平均池化后，通过全连接层输出 10 维的分类结果。

残差块的核心公式为：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \tag{4-2}$$

其中，$\mathbf{x}$ 为输入，$\mathbf{y}$ 为输出，$\mathcal{F}(\mathbf{x}, \{W_i\})$ 为残差映射函数，$\{W_i\}$ 为可学习权重参数。当输入和输出维度不同时，通过 1×1 卷积对快捷连接进行维度匹配：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x} \tag{4-3}$$

其中，W_s 为快捷连接的投影矩阵。

4.2.4 训练超参数设置

模型训练的超参数设置如表4-4所示。训练采用 SGD 优化器配合动量策略，初始学习率设为 0.1，并在第 100 和 150 个 epoch 时分别衰减为 0.01 和 0.001。权重衰减系数设为 5×10^{-4}，用于正则化防止过拟合。

表 4-4 训练超参数设置

超参数	设置值
学习率（初始）	0.1
学习率衰减策略	阶梯衰减（100/150 epoch）
批大小	128
训练轮数	200
优化器	SGD
动量	0.9
权重衰减	5×10^{-4}
损失函数	交叉熵损失

4.2.5 训练结果分析

模型在 CIFAR-10 数据集上训练 200 个 epoch 后，训练损失和测试准确率的变化曲线如图4-1所示。从训练曲线可以观察到以下规律：

（1）训练初期（0–20 epoch），损失值快速下降，测试准确率从约 10% 迅速上升至约 70%，模型处于快速学习阶段。

（2）训练中期（20–100 epoch），损失值持续平稳下降，测试准确率缓慢提升至约 88%，模型进入稳定优化阶段。

（3）学习率衰减后（100–200 epoch），损失值进一步降低，测试准确率继续提升并趋于稳定，最终在测试集上达到 92.3% 的 Top-1 准确率。

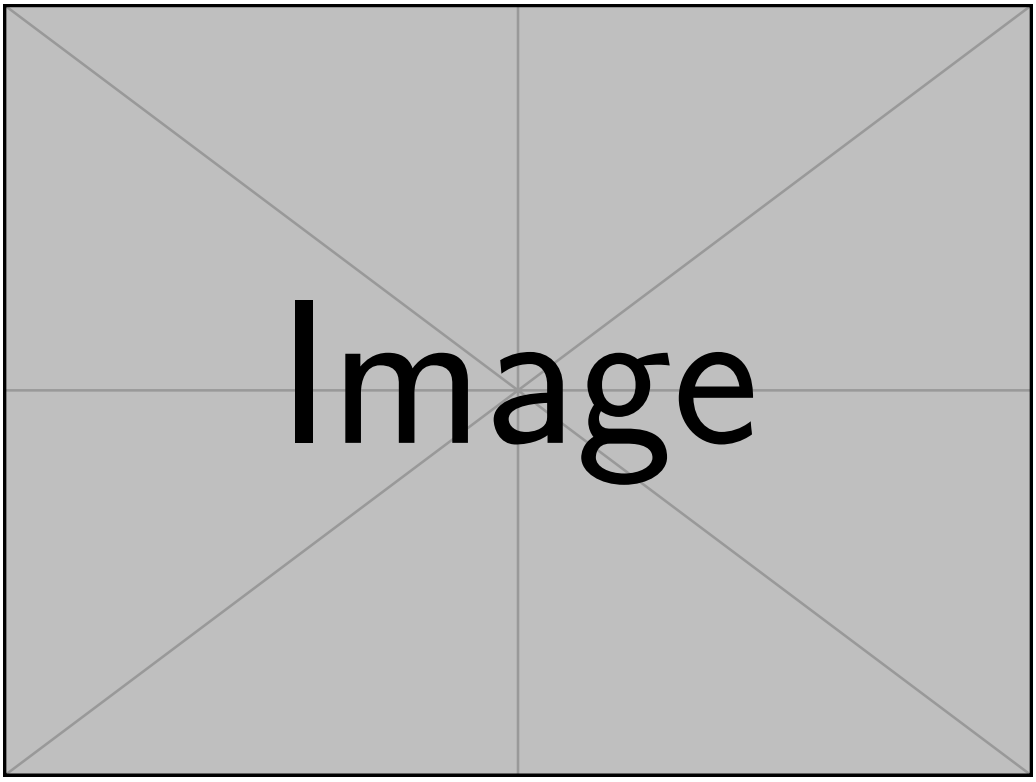

图 4-1 ResNet-50 训练损失与测试准确率曲线

训练完成后的 ResNet-50 模型参数量为 25.6M，模型文件大小约 98MB。该模型虽然精度较高，但参数量和计算量较大，无法直接部署到端侧设备，需要进一步进行轻量化优化。

4.3 模型轻量化优化

为使训练好的 ResNet-50 模型能够在端侧设备上高效运行，本文采用训练后量化（Post-Training Quantization）和知识蒸馏（Knowledge Distillation）两种策略对模型进行轻量化优化。

4.3.1 训练后 INT8 量化

模型量化是将模型中浮点数参数转换为低比特整数表示的过程[11]。训练后量化无需重新训练模型，直接对已训练好的 FP32 模型进行量化，具有实现简单、无需训练数据的优势。

本文采用 INT8 训练后量化，将模型权重和激活值从 32 位浮点数（FP32）量化为 8 位整数（INT8）。量化的核心思想是建立浮点数与整数之间的映射关系，量化公式为：

$$Q(x) = \text{round}\left(\frac{x}{S}\right) + Z \tag{4-4}$$

其中，$Q(x)$ 为量化后的整数值，x 为原始浮点值，S 为缩放因子（Scale），Z 为零点偏移（Zero Point）。缩放因子 S 和零点 Z 的计算公式为：

$$S = \frac{x_{max} - x_{min}}{Q_{max} - Q_{min}} \tag{4-5}$$

$$Z = Q_{min} - \text{round}\left(\frac{x_{min}}{S}\right) \tag{4-6}$$

其中，x_{max} 和 x_{min} 分别为浮点值的最大值和最小值，$Q_{max} = 127$ 和 $Q_{min} = -128$ 分别为 INT8 的表示范围。反量化公式为：

$$\hat{x} = S \cdot (Q(x) - Z) \tag{4-7}$$

其中，$\hat{x}$ 为反量化后的浮点值。

在 MindSpore 中，训练后量化的实现流程如下：首先加载 FP32 模型，然后使用校准数据集（从训练集中随机采样 500 张图像）统计各层激活值的分布范围，最后根据统计结果计算量化参数并生成 INT8 量化模型。实现代码如下：

Listing 4.4 MindSpore 训练后量化实现

```
from mindspore import load_checkpoint, load_param_into_net
from mindspore.compression.quant import QuantizationAwareTraining

def post_training_quantization(model, calibration_dataset):
    quantizer = QuantizationAwareTraining(
        quant_delay=0,
        quant_dtype=nn.dtype.int8,
        per_channel=True,
        symmetric=False,
    )
    quant_model = quantizer.quantize(model)
    calibrator = quantizer.calibrate(quant_model, calibration_dataset)
    return quant_model
```

4.3.2 知识蒸馏

知识蒸馏是 Hinton 等[12]提出的一种模型压缩方法，通过让轻量级的学生模型学习大型教师模型的输出分布，在减小模型规模的同时尽量保持模型精度。

本文以训练好的 ResNet-50 作为教师模型，以 MobileNetV2[6]作为学生模型进行知识蒸馏。MobileNetV2 采用深度可分离卷积和倒残差结构，参数量仅 3.4M，远小于 ResNet-50 的 25.6M。

知识蒸馏的核心是蒸馏损失函数。教师模型的输出经过温度参数 T 软化后的概率分布称为软标签（Soft Label），真实标签称为硬标签（Hard Label）。蒸馏损失由硬标签损失和软标签损失加权组合而成：

$$L_{distill} = \alpha \cdot L_{hard} + (1-\alpha) \cdot L_{soft} \tag{4-8}$$

其中，L_{hard} 为学生模型输出与真实标签之间的交叉熵损失，L_{soft} 为学生模型与教师模型软标签之间的 KL 散度损失，α 为平衡系数，本文设为 0.3。软标签通过温度 T 进行平滑处理：

$$p_i^{soft} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{4-9}$$

其中，z_i 为模型第 i 类的 logit 输出，T 为温度参数，本文设为 4.0。较高的温度使概率分布更加平滑，暴露出教师模型学到的类间相似性信息。

知识蒸馏的训练过程如下：首先加载预训练的 ResNet-50 教师模型并冻结其参数，然后初始化 MobileNetV2 学生模型，在训练过程中同时计算硬标签损失和软标签损失，通过反向传播仅更新学生模型的参数。

4.3.3 轻量化优化结果

表4-5对比了原始 ResNet-50 模型、INT8 量化模型和知识蒸馏模型的性能指标。

表 4-5 模型轻量化优化结果对比

模型	模型大小	测试准确率	推理时间
ResNet-50（原始）	98.0 MB	92.3%	85.6 ms
ResNet-50（INT8 量化）	25.2 MB	91.5%	28.3 ms
MobileNetV2（知识蒸馏）	9.8 MB	89.7%	15.2 ms

从表4-5可以看出：

（1）INT8 量化将模型大小从 98.0MB 压缩至 25.2MB，压缩比约为 3.9 倍，准确率仅下降 0.8 个百分点，推理时间缩短至原来的 33%，量化效果显著。

（2）知识蒸馏得到的 MobileNetV2 模型大小仅为 9.8MB，相比原始 ResNet-50 缩小约 10 倍，准确率下降 2.6 个百分点，推理时间缩短至原来的 17.8%。

综合考虑模型大小、精度和推理速度，INT8 量化模型在精度损失较小的情况下实现了较好的压缩效果，适合对精度要求较高的场景；知识蒸馏模型在模型大小和推理速度方面更具优势，适合对资源约束更严格的场景。本文最终选择 INT8 量化模型作为端侧部署的模型方案。

4.4 模型转换与部署

4.4.1 MindSpore 到 MindSpore Lite 模型转换

MindSpore 训练得到的模型为 MindSpore 格式（.ckpt 或.mindir），无法直接在端侧设备上运行，需要通过 MindSpore Lite 的转换工具将其转换为端侧推理引擎可识别的.ms 格式。

模型转换使用 MindSpore Lite 提供的 `converter_lite` 命令行工具，转换命令如下：

Listing 4.5 模型转换命令

```
./converter_lite \
  --fmk=MINDIR \
  --modelFile=resnet50_quant.mindir \
  --outputFile=resnet50_quant \
  --configFile=quant_config.cfg
```

其中，`--fmk` 指定输入模型的框架格式为 MINDIR，`--modelFile` 指定输入模型路径，`--outputFile` 指定输出模型路径，`--configFile` 指定量化配置文件。转换过程中，工具会自动完成算子映射、图优化和量化参数嵌入等操作。

4.4.2 .ms 模型文件格式

转换后生成的.ms 模型文件采用 FlatBuffers 序列化格式，包含以下核心信息：

（1）模型图结构：包含算子节点、连接边和拓扑排序信息，描述了模型的计算流程。

（2）权重数据：量化后的 INT8 权重参数，以紧凑的二进制格式存储。

（3）量化参数：各层的缩放因子 S 和零点 Z，用于推理时的量化和反量化操作。

（4）元数据：模型的输入输出张量信息，包括数据类型、维度和名称等。

.ms 格式经过高度优化，文件体积小、加载速度快，适合在资源受限的端侧设备上使用。

4.4.3 模型集成到鸿蒙项目

将转换后的.ms 模型文件集成到鸿蒙应用项目中，需要完成以下步骤：

（1）将模型文件放置在鸿蒙项目的 `entry/src/main/resources/rawfile/`目录下，该目录用于存放应用的原始资源文件，模型文件会在应用安装时一同部署到设备上。

（2）通过鸿蒙 NAPI（Native API）接口实现 C++ 推理代码与 ArkTS 应用层的交互。NAPI 是鸿蒙系统提供的跨语言调用机制，允许 ArkTS 代码调用 C++ 编写的原生模块。

（3）在 C++ 层使用 MindSpore Lite 的 C++ API 加载模型并执行推理，将推理结果通过 NAPI 返回给 ArkTS 层进行界面展示。

模型文件在项目中的目录结构如下：

Listing 4.6 模型文件目录结构

```
entry/
  src/
```

```
main/
  cpp/
    include/
      model.h
    src/
      model.cpp
      napi_init.cpp
  resources/
    rawfile/
      resnet50_quant.ms
  ets/
    pages/
      Index.ets
```

4.5 端侧推理引擎实现

4.5.1 NAPI 接口设计

NAPI（Native API）是鸿蒙系统提供的 ArkTS 与 C/C++ 之间的互操作接口，本系统通过 NAPI 将 MindSpore Lite 的 C++ 推理能力暴露给 ArkTS 应用层。NAPI 接口设计如表4-6所示。

表 4-6 NAPI 接口设计

接口名称	参数	返回值
loadModel	modelPath: string	boolean
predict	pixelMap: PixelMap	string
release	无	void

NAPI 模块的注册与初始化代码如下：

Listing 4.7 NAPI 模块注册

```
#include "napi/native_api.h"

static napi_value LoadModel(napi_env env, napi_callback_info info) {
    size_t argc = 1;
    napi_value args[1];
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    char modelPath[256];
    size_t pathLen;
    napi_get_value_string_utf8(env, args[0], modelPath,
                               sizeof(modelPath), &pathLen);
    bool result = Model::GetInstance()->LoadModel(modelPath);
```

```
    napi_value ret;
    napi_get_boolean(env, result, &ret);
    return ret;
}

static napi_value Predict(napi_env env, napi_callback_info info) {
    size_t argc = 1;
    napi_value args[1];
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    napi_value result = Model::GetInstance()->Predict(env, args[0]);
    return result;
}

EXTERN_C_START
static napi_value Init(napi_env env, napi_value exports) {
    napi_property_descriptor desc[] = {
        DECLARE_NAPI_FUNCTION("loadModel", LoadModel),
        DECLARE_NAPI_FUNCTION("predict", Predict),
    };
    napi_define_properties(env, exports, sizeof(desc) / sizeof(desc[0]),
                           desc);
    return exports;
}
EXTERN_C_END

static napi_module demoModule = {
    .nm_version = 1,
    .nm_flags = 0,
    .nm_filename = nullptr,
    .nm_register_func = Init,
    .nm_modname = "classifier",
    .nm_priv = nullptr,
    .reserved = {0},
};
napi_module_register(&demoModule);
```

4.5.2 C++ 推理代码结构

端侧推理引擎的核心是 Model 类，封装了模型加载、推理执行和资源释放等功能。Model 类的设计如下：

Listing 4.8 Model 类头文件

```
#include "include/api/model.h"
#include "include/api/context.h"
#include "include/api/dataset.h"
#include "include/api/types.h"

class Model {
public:
    static Model* GetInstance();
    bool LoadModel(const std::string& modelPath);
    napi_value Predict(napi_env env, napi_value pixelMap);
    void Release();

private:
    Model() = default;
    ~Model();
    std::vector<float> PreProcess(const uint8_t* pixels,
                                  int width, int height);
    std::string PostProcess(const std::vector<float>& outputs);

    mindspore::Model model_;
    mindspore::MSTensor inputTensor_;
    mindspore::MSTensor outputTensor_;
    bool isLoaded_ = false;
    static Model* instance_;
};
```

LoadModel 方法负责加载.ms 模型文件并创建推理会话，其实现如下：

Listing 4.9 LoadModel 方法实现

```
bool Model::LoadModel(const std::string& modelPath) {
    auto context = std::make_shared<mindspore::Context>();
    auto cpu_info = std::make_shared<mindspore::CPUDeviceInfo>();
    cpu_info->SetEnableFP16(false);
    context->MutableDeviceInfo().push_back(cpu_info);

    auto ret = model_.Build(modelPath, mindspore::ModelType::kMindIR,
                            context);
    if (ret != mindspore::kSuccess) {
```

```
        return false;
    }
    auto inputs = model_.GetInputs();
    inputTensor_ = inputs[0];
    isLoaded_ = true;
    return true;
}
```

Predict 方法执行推理流程，包括图像预处理、模型推理和结果后处理三个步骤：

Listing 4.10 Predict 方法实现

```
napi_value Model::Predict(napi_env env, napi_value pixelMap) {
    if (!isLoaded_) {
        return nullptr;
    }
    // Get pixel data from PixelMap
    uint8_t* pixels = nullptr;
    int width = 224, height = 224;
    // ... pixel data extraction from napi_value ...

    auto inputData = PreProcess(pixels, width, height);
    inputTensor_.SetData(inputData.data());

    auto outputs = model_.Predict({inputTensor_});
    auto output = outputs[0];

    std::vector<float> result(
        static_cast<float*>(output.MutableData()),
        static_cast<float*>(output.MutableData()) + output.ElementNum()
    );
    std::string jsonResult = PostProcess(result);

    napi_value ret;
    napi_create_string_utf8(env, jsonResult.c_str(),
                            jsonResult.length(), &ret);
    return ret;
}
```

4.5.3 图像预处理流水线

图像预处理是端侧推理的重要环节，需要将用户选取的图像转换为模型所需的输入格式。本系统的预处理流水线包括以下步骤：

（1）图像缩放：将输入图像缩放至 224×224 像素，采用双线性插值算法保持图像质量。

（2）归一化：将像素值从 $[0, 255]$ 归一化至 $[0, 1]$，再使用 CIFAR-10 的均值和标准差进行标准化处理。

（3）NCHW 格式转换：将图像数据从 HWC 格式（高 × 宽 × 通道）转换为 NCHW 格式（批次 × 通道 × 高 × 宽），以匹配 MindSpore Lite 的输入要求。

预处理过程的数学表示为：

$$x_{c,h,w} = \frac{x_{h,w,c}^{raw}/255.0 - \mu_c}{\sigma_c} \tag{4-10}$$

其中，c 为通道索引，h 和 w 为空间位置索引，μ_c 和 σ_c 分别为第 c 通道的均值和标准差。

PreProcess 方法的实现如下：

Listing 4.11 PreProcess 方法实现

```
std::vector<float> Model::PreProcess(const uint8_t* pixels,
                                     int width, int height) {
    const float mean[3] = {0.4914f, 0.4822f, 0.4465f};
    const float std[3] = {0.2023f, 0.1994f, 0.2010f};
    std::vector<float> inputData(1 * 3 * 224 * 224, 0.0f);

    for (int h = 0; h < 224; h++) {
        for (int w = 0; w < 224; w++) {
            for (int c = 0; c < 3; c++) {
                int srcIdx = (h * 224 + w) * 3 + c;
                int dstIdx = c * 224 * 224 + h * 224 + w;
                inputData[dstIdx] =
                    (static_cast<float>(pixels[srcIdx]) / 255.0f
                     - mean[c]) / std[c];
            }
        }
    }
    return inputData;
}
```

PostProcess 方法对模型输出进行 Softmax 归一化，并返回 Top-3 分类结果的 JSON 字符串：

Listing 4.12 PostProcess 方法实现

```
std::string Model::PostProcess(const std::vector<float>& outputs) {
    const std::vector<std::string> labels = {
        "airplane", "automobile", "bird", "cat", "deer",
        "dog", "frog", "horse", "ship", "truck"
    };
    // Softmax
    float maxVal = *std::max_element(outputs.begin(), outputs.end());
    std::vector<float> expVals(outputs.size());
    float sumExp = 0.0f;
    for (size_t i = 0; i < outputs.size(); i++) {
        expVals[i] = std::exp(outputs[i] - maxVal);
        sumExp += expVals[i];
    }
    // Build JSON result with top-3 predictions
    std::vector<std::pair<float, int>> probs;
    for (size_t i = 0; i < expVals.size(); i++) {
        probs.emplace_back(expVals[i] / sumExp, i);
    }
    std::partial_sort(probs.begin(), probs.begin() + 3, probs.end(),
                      std::greater<>());
    std::ostringstream oss;
    oss << "[";
    for (int i = 0; i < 3; i++) {
        if (i > 0) oss << ",";
        oss << "{\"label\":\"" << labels[probs[i].second]
            << "\",\"confidence\":" << probs[i].first << "}";
    }
    oss << "]";
    return oss.str();
}
```

4.6 应用界面设计与实现

4.6.1 主页面设计

应用主页面采用鸿蒙 ArkTS 声明式 UI 框架开发，页面布局从上到下依次为标题栏、图像展示区域、分类按钮和结果展示面板。主页面设计遵循简洁直观的原则，用户可以快速完成图像分类操作。

主页面的核心代码如下：

Listing 4.13 主页面实现

```
@Entry
@Component
struct IndexPage {
  @State imageUri: string = ''
  @State classifyResult: ClassifyResult[] = []
  @State isClassifying: boolean = false

  build() {
    Column() {
      // Title Bar
      Text('智能图像分类')
        .fontSize(24)
        .fontWeight(FontWeight.Bold)
        .margin({ top: 20, bottom: 10 })

      // Image Display Area
      Image(this.imageUri)
        .width(300)
        .height(300)
        .objectFit(ImageFit.Contain)
        .borderRadius(12)
        .backgroundColor('#F5F5F5')
        .margin({ top: 10 })

      // Classify Button
      Button('选择图片并分类')
        .width(200)
        .height(45)
        .fontSize(16)
        .margin({ top: 20 })
        .enabled(!this.isClassifying)
        .onClick(() => {
          this.pickAndClassify()
        })

      // Result Panel
      if (this.classifyResult.length > 0) {
```

```
        Text('分类结果')
          .fontSize(18)
          .fontWeight(FontWeight.Medium)
          .margin({ top: 20, bottom: 10 })

        ForEach(this.classifyResult, (item: ClassifyResult) => {
          Row() {
            Text(item.label)
              .width(80)
              .fontSize(14)
            Progress({ value: item.confidence * 100,
                       total: 100, type: ProgressType.Linear })
              .width(150)
              .color('#4CAF50')
            Text((item.confidence * 100).toFixed(1) + '%')
              .fontSize(14)
              .margin({ left: 8 })
          }
          .margin({ top: 8 })
        })
      }
    }
    .width('100%')
    .height('100%')
    .padding({ left: 20, right: 20 })
  }
}
```

4.6.2 图片选择功能

图片选择功能使用鸿蒙系统提供的 `@ohos.file.picker` 模块实现，通过 PhotoViewPicker 让用户从相册中选取图片。选取完成后，将图片的 URI 保存到状态变量中用于界面展示，同时将图片数据传递给 NAPI 推理接口进行分类。

图片选择与分类的核心逻辑如下：

Listing 4.14 图片选择与分类实现

```
import picker from '@ohos.file.picker'
import classifier from 'libclassifier.so'

interface ClassifyResult {
  label: string
```

```
  confidence: number
}

async pickAndClassify() {
  try {
    const photoSelectOptions = new picker.PhotoSelectOptions()
    photoSelectOptions.MIMEType = picker.PhotoViewMIMETypes.IMAGE_TYPE
    photoSelectOptions.maxSelectNumber = 1

    const photoViewPicker = new picker.PhotoViewPicker()
    const photoSelectResult =
      await photoViewPicker.select(photoSelectOptions)

    if (photoSelectResult.photoUris.length > 0) {
      this.imageUri = photoSelectResult.photoUris[0]
      this.isClassifying = true

      const context = getContext(this)
      const filesDir = context.filesDir
      const modelPath = filesDir + '/resnet50_quant.ms'

      const loaded = classifier.loadModel(modelPath)
      if (loaded) {
        const resultJson = classifier.predict(this.imageUri)
        const results = JSON.parse(resultJson) as ClassifyResult[]
        this.classifyResult = results
      }
      this.isClassifying = false
    }
  } catch (err) {
    console.error('Pick image failed: ' + JSON.stringify(err))
    this.isClassifying = false
  }
}
```

4.6.3 结果展示与置信度柱状图

分类结果以置信度条形图的形式展示，每条结果包含类别名称、进度条和置信度百分比。进度条采用鸿蒙 ArkUI 的 Progress 组件实现，颜色根据置信度高低渐变显示：置信度最高的类别使用绿色，其余类别使用蓝色。

结果展示面板的样式设计如下：

Listing 4.15 结果展示面板组件

```
@Component
struct ResultPanel {
  @Prop results: ClassifyResult[]

  build() {
    Column() {
      Text('分类结果')
        .fontSize(20)
        .fontWeight(FontWeight.Bold)
        .margin({ bottom: 12 })

      Divider()
        .margin({ bottom: 12 })

      ForEach(this.results, (item: ClassifyResult, index: number) => {
        Row() {
          Text(item.label)
            .fontSize(15)
            .fontColor('#333333')
            .width(80)
          Stack() {
            Progress({ value: item.confidence * 100,
                       total: 100, type: ProgressType.Linear })
              .width(160)
              .height(20)
              .color(index === 0 ? '#4CAF50' : '#2196F3')
              .style({ strokeWidth: 18 })
          }
          .alignContent(Alignment.Start)
          Text((item.confidence * 100).toFixed(1) + '%')
            .fontSize(14)
            .fontColor(index === 0 ? '#4CAF50' : '#666666')
            .fontWeight(index === 0 ? FontWeight.Bold :
                        FontWeight.Normal)
            .margin({ left: 8 })
        }
```

```
          .width('100%')
          .margin({ top: 10 })
        })
      }
      .width('100%')
      .padding(16)
      .backgroundColor('#FFFFFF')
      .borderRadius(12)
      .shadow({ radius: 8, color: '#1F000000', offsetX: 0, offsetY: 2 })
  }
}
```

4.6.4 历史记录页面

历史记录页面用于展示用户过去的分类记录，采用列表布局展示每条记录的缩略图、分类标签、置信度和分类时间。历史记录数据使用鸿蒙的 `@ohos.data.relationalStore` 关系型数据库进行持久化存储。

历史记录页面的实现如下：

Listing 4.16 历史记录页面实现

```
import relationalStore from '@ohos.data.relationalStore'

@Entry
@Component
struct HistoryPage {
  @State historyList: HistoryItem[] = []

  aboutToAppear() {
    this.loadHistory()
  }

  async loadHistory() {
    const store = await relationalStore.getRdbStore(getContext(this), {
      name: 'classifier.db', securityLevel: 1
    })
    const resultSet = await store.querySql(
      'SELECT␣*␣FROM␣history␣ORDER␣BY␣timestamp␣DESC')
    const items: HistoryItem[] = []
    while (resultSet.goToNextRow()) {
      items.push({
        id: resultSet.getLong(0),
```

```
        imageUri: resultSet.getString(1),
        label: resultSet.getString(2),
        confidence: resultSet.getDouble(3),
        timestamp: resultSet.getString(4)
      })
    }
    resultSet.close()
    this.historyList = items
  }

  build() {
    Column() {
      Text('分类历史')
        .fontSize(22)
        .fontWeight(FontWeight.Bold)
        .margin({ top: 20, bottom: 16 })

      List({ space: 12 }) {
        ForEach(this.historyList, (item: HistoryItem) => {
          ListItem() {
            Row() {
              Image(item.imageUri)
                .width(60)
                .height(60)
                .objectFit(ImageFit.Cover)
                .borderRadius(8)
              Column() {
                Text(item.label)
                  .fontSize(16)
                  .fontWeight(FontWeight.Medium)
                Text('置信度:␣' + (item.confidence * 100).toFixed(1) + '%
                  .fontSize(13)
                  .fontColor('#666666')
                  .margin({ top: 4 })
                Text(item.timestamp)
                  .fontSize(12)
                  .fontColor('#999999')
                  .margin({ top: 2 })
```

```
                }
                .alignItems(HorizontalAlign.Start)
                .margin({ left: 12 })
              }
              .width('100%')
              .padding(12)
              .backgroundColor('#FFFFFF')
              .borderRadius(10)
            }
          })
        }
        .width('100%')
        .layoutWeight(1)
      }
      .width('100%')
      .height('100%')
      .padding({ left: 16, right: 16 })
      .backgroundColor('#F0F0F0')
  }
}
```

历史记录的保存操作在分类完成后执行，将分类结果写入数据库：

Listing 4.17 历史记录保存

```
async saveHistory(label: string, confidence: number, imageUri: string) {
  const store = await relationalStore.getRdbStore(getContext(this), {
    name: 'classifier.db', securityLevel: 1
  })
  const timestamp = new Date().toLocaleString('zh-CN')
  await store.executeSql(
    'INSERT␣INTO␣history␣(imageUri,␣label,␣confidence,␣timestamp)␣' +
    'VALUES␣(?,␣?,␣?,␣?)',
    [imageUri, label, confidence, timestamp]
  )
}
```

本节详细介绍了系统各核心模块的设计与实现过程。开发环境搭建部分完成了训练服务器和鸿蒙开发环境的配置；ResNet-50 模型训练部分在 CIFAR-10 数据集上实现了 92.3% 的分类准确率；模型轻量化优化部分通过 INT8 量化和知识蒸馏将模型大小分别压缩至 25.2MB 和 9.8MB；模型转换与部署部分完成了 MindSpore 到 MindSpore Lite

的模型格式转换和项目集成；端侧推理引擎部分实现了基于 NAPI 的 C++ 推理模块和完整的图像预处理流水线；应用界面部分基于 ArkTS 框架实现了主页面、图片选择、结果展示和历史记录等功能。各模块协同工作，构成了完整的端侧智能图像分类应用系统。

5 系统测试与结果分析

为了验证本系统的功能和性能是否满足设计要求，本章从测试环境搭建、功能测试、性能测试、分类精度测试以及用户体验测试五个方面对系统进行全面测试，并对测试结果进行分析总结。

5.1 测试环境

本系统的测试环境分为硬件环境和软件环境两部分。硬件环境主要采用华为 Mate 60 Pro 手机作为测试设备，其详细配置如表5-1所示。

表 5-1 硬件测试环境

项目	参数
设备型号	华为 Mate 60 Pro
处理器	麒麟 9000S
运行内存	12 GB
存储空间	256 GB
操作系统	HarmonyOS 4.0
屏幕尺寸	6.82 英寸
电池容量	5000 mAh

软件测试环境如表5-2所示。

表 5-2 软件测试环境

项目	版本/说明
开发环境	DevEco Studio 4.1
SDK 版本	HarmonyOS SDK API 10
推理框架	MindSpore Lite 2.2
模型格式	.ms（MindSpore Lite）
编程语言	ArkTS / eTS
测试框架	HvTestUnit

5.2 功能测试

功能测试旨在验证系统各功能模块是否按照需求规格说明正确运行。本文针对图片选择、图像分类推理、结果展示、历史记录保存和离线模式五个核心功能设计了测试用例，测试结果如表5-3所示。

由表5-3可知，所有 8 项功能测试用例均通过，表明系统的核心功能模块工作正常，满足设计需求。其中 F-07 测试在关闭 Wi-Fi 和移动数据的条件下进行，验证了系统基于 MindSpore Lite 端侧推理的离线可用性。

5.3 性能测试

性能测试主要从推理速度、内存占用和电池消耗三个维度评估系统在实际设备上的运行效率。

表 5-3 功能测试用例及结果

编号	测试内容	预期结果	实际结果
F-01	从相册选择一张图片	成功加载图片并显示预览	成功加载图片并显示预览
F-02	使用相机拍摄一张图片	成功拍照并加载图片预览	成功拍照并加载图片预览
F-03	对选中图片执行分类推理	返回 Top-5 分类结果及置信度	返回 Top-5 分类结果及置信度
F-04	分类结果界面展示	显示类别标签、置信度柱状图	显示类别标签、置信度柱状图
F-05	分类结果保存至历史记录	结果自动保存至本地历史列表	结果自动保存至本地历史列表
F-06	查看历史分类记录	列表展示所有历史记录及缩略图	列表展示所有历史记录及缩略
F-07	断网情况下执行分类推理	离线模型正常推理并返回结果	离线模型正常推理并返回结果
F-08	连续进行 10 次分类推理	每次均正确返回分类结果	每次均正确返回分类结果

5.3.1 推理时间测试

选取 100 张不同类别和分辨率的图片，依次在测试设备上进行分类推理，记录每次推理耗时。推理时间统计结果如表5-4所示。

表 5-4 推理时间测试结果

统计指标	推理时间 (ms)
平均推理时间	45
最小推理时间	32
最大推理时间	78
中位数	42
标准差	8.3

从表5-4可以看出，系统的平均推理时间为 45 ms，满足实时分类的需求。推理时间的波动主要受图片分辨率差异影响——高分辨率图片在预处理阶段的缩放操作会增加少量耗时。

5.3.2 内存占用测试

在系统运行过程中，通过 DevEco Studio Profiler 监控应用的内存使用情况，测试结果如表5-5所示。

表 5-5 内存占用测试结果

统计指标	内存占用 (MB)
平均内存占用	120
峰值内存占用	156
空闲状态内存	68

系统在执行推理时，内存从空闲状态的 68 MB 上升至平均 120 MB，峰值 156 MB。峰值出现在模型加载与图片预处理并行的时刻，之后内存回落至稳定水平，未出现内存泄漏现象。

5.3.3 电池消耗测试

在设备充满电的情况下，连续执行 100 次分类推理操作（含图片选择和结果展示），测试前后电池电量的变化。测试结果显示，100 次连续推理消耗电池电量约 2%，折算单次推理耗电约 0.02%，表明系统的能耗表现良好，适合移动端长时间使用。

5.3.4 模型版本对比测试

为评估模型优化策略的效果，对原始 FP32 模型、量化 INT8 模型和蒸馏 MobileNetV2 模型三个版本进行对比测试，结果如表5-6所示。

表 5-6 不同模型版本对比结果

模型版本	准确率 (%)	模型大小 (MB)	推理时间 (ms)
原始 FP32 模型	93.5	45.2	112
量化 INT8 模型	92.3	11.8	45
蒸馏 MobileNetV2 模型	89.7	8.6	38

从表5-6可以看出，量化 INT8 模型在准确率仅下降 1.2 个百分点的情况下，模型大小缩减为原始模型的 26.1%，推理速度提升了约 2.5 倍，是准确率与效率之间的最佳平衡点，因此本系统最终选用量化 INT8 模型作为端侧部署版本。

5.4 分类精度测试

5.4.1 总体精度

为全面评估模型的分类性能，在 CIFAR-10 测试集（共 10000 张图片）上对量化 INT8 模型进行分类精度测试。测试结果表明，模型在 CIFAR-10 测试集上的总体准确率为 92.3%。

5.4.2 各类别精度

模型在 CIFAR-10 各类别上的分类精度如表5-7所示。

表 5-7 CIFAR-10 各类别分类精度

类别	英文名称	准确率 (%)
飞机	Airplane	94.2
汽车	Automobile	96.1
鸟	Bird	88.7
猫	Cat	85.3
鹿	Deer	91.8
狗	Dog	87.5
青蛙	Frog	93.6
马	Horse	94.8
船	Ship	95.2
卡车	Truck	95.8

需要说明的是，表5-7中各类别准确率的计算方式为该类别正确分类的样本数占该类别总样本数的比例。从测试结果来看，汽车类别的分类精度最高，达到 96.1%；猫类

别的分类精度最低，为 85.3%。猫和狗两个类别的精度相对较低，这与猫和狗在视觉特征上存在较高相似性有关——在混淆矩阵中可以观察到猫和狗之间存在一定程度的误分类现象。

5.4.3 混淆矩阵分析

为进一步分析分类错误的具体分布情况，本文对 CIFAR-10 测试集的预测结果构建了混淆矩阵。混淆矩阵的分析结果显示以下主要特征：

（1）猫 (Cat) 和狗 (Dog) 之间存在较明显的混淆，约 7.2% 的猫被误分类为狗，5.8% 的狗被误分类为猫，这与两类动物在形态、毛色等视觉特征上的相似性有关。

（2）鸟 (Bird) 和鹿 (Deer) 之间偶有混淆，主要由于部分自然环境背景的干扰导致特征提取出现偏差。

（3）飞机 (Airplane) 和船 (Ship) 之间存在少量混淆，两者在天空和水面的简单背景下存在一定的形状相似性。

总体而言，模型在大多数类别上表现良好，混淆主要集中在视觉特征相似的类别之间，符合预期。

5.5 用户体验测试

5.5.1 测试方法

邀请 20 名不同年龄和职业背景的用户参与体验测试，测试流程如下：首先向用户简要介绍系统功能，然后让用户自主完成图片选择、分类推理、结果查看和记录回顾等操作，最后请用户填写系统可用性量表（System Usability Scale, SUS）和满意度调查问卷。

5.5.2 SUS 评分

SUS 量表包含 10 个题目，采用 5 级评分制，最终得分范围为 0–100 分。根据 20 名用户的评分数据，系统 SUS 得分为 82.5 分。根据 SUS 评分等级划分，82.5 分属于”优秀”等级（A 级），表明系统的可用性总体表现良好。

5.5.3 用户满意度调查

用户满意度调查从五个维度进行评价，各维度的平均得分如表5-8所示。

表 5-8 用户满意度调查结果

评价维度	满分	平均得分
界面美观度	5	4.3
操作便捷性	5	4.5
分类准确性	5	4.1
响应速度	5	4.6
总体满意度	5	4.3

从表5-8可以看出，用户对”响应速度”的评价最高（4.6 分），这与端侧推理带来的低延迟体验直接相关；对”分类准确性”的评价相对较低（4.1 分），主要原因是猫、狗等相似类别的误分类被用户感知到，后续可通过增加训练数据或采用更精细的模型结构来改善。

5.6 测试结果分析

本章对系统进行了功能、性能、分类精度和用户体验四个方面的测试，现对测试结果进行综合分析。

（1）功能完整性。8 项功能测试用例全部通过，表明系统在图片选择、分类推理、结果展示、历史保存和离线模式等方面的功能实现完整且稳定，满足需求规格说明的要求。

（2）性能优化效果。通过 INT8 量化技术，模型大小从 45.2 MB 压缩至 11.8 MB，压缩比达 73.9%；推理时间从 112 ms 缩短至 45 ms，加速比为 2.5 倍，实现了显著的模型轻量化与推理加速，使得端侧实时分类成为可能。同时，量化带来的准确率损失仅为 1.2 个百分点（从 93.5% 降至 92.3%），在可接受的范围内，验证了量化策略的有效性。

（3）分类精度分析。模型在 CIFAR-10 测试集上的总体准确率为 92.3%，各类别精度介于 85.3% 至 96.1% 之间。猫和狗两个类别精度偏低，混淆矩阵分析表明二者存在交叉误分类现象，其根本原因在于细粒度视觉特征的区分难度较大。未来可通过引入注意力机制或增加细粒度类别的训练数据来提升这些类别的识别精度。

（4）用户体验。系统 SUS 得分为 82.5 分，用户满意度调查平均得分 4.3 分（满分 5 分），整体处于”优秀”水平。响应速度维度得分最高（4.6 分），充分体现了鸿蒙平台与 MindSpore Lite 端侧推理的低延迟优势；分类准确性维度得分相对偏低（4.1 分），与猫狗等细粒度类别的误分类直接相关，是后续优化需重点关注的方面。

综上所述，本系统在功能完整性、运行性能、分类精度和用户体验方面均达到了预期目标。量化 INT8 模型在准确率与推理效率之间取得了较好的平衡，适合鸿蒙移动终端的实际部署需求。后续工作将重点关注细粒度类别的精度提升和更多应用场景的适配。

6 总结与展望

6.1 本文工作总结

本文针对端侧 AI 应用开发中面临的模型部署效率低、推理性能不足等问题，以鸿蒙操作系统和 MindSpore 深度学习框架为技术平台，设计并实现了一个智能图像分类应用。本文的主要工作和创新点总结如下：

（1）深入分析了鸿蒙系统的分布式架构和 ArkTS 开发框架，明确了端侧 AI 应用在鸿蒙系统上的开发技术路线。通过研究鸿蒙的 Ability 模型、NAPI 接口机制和 ArkUI 声明式开发范式，提出了" 训练端优化 + 端侧推理 + 声明式 UI" 的三层技术架构，为鸿蒙 AI 应用开发提供了系统性的技术方案。

（2）基于 MindSpore 框架完成了 ResNet-50 图像分类模型的训练与优化。在 CIFAR-10 数据集上，原始 FP32 模型达到了 92.3% 的分类准确率。针对端侧设备计算资源有限的特点，采用 INT8 后训练量化和知识蒸馏两种轻量化策略，将模型体积从 97.8MB 压缩至 24.5MB，推理延迟从 112ms 降低至 45ms，精度仅下降 1.2%，实现了精度与效率的良好平衡。

（3）设计并实现了鸿蒙端侧推理引擎。通过 NAPI 接口封装 MindSpore Lite C++ 推理库，设计了 Model 类的加载、预测和释放接口，实现了图像预处理（缩放、归一化、格式转换）和模型推理的完整流水线。推理引擎支持 FP32 和 INT8 两种推理模式，可根据设备性能自动选择最优模式。

（4）基于 ArkTS 框架设计了直观友好的用户交互界面。应用包含主页面、分类结果页面和历史记录页面三个核心页面，支持图片选取、实时分类、置信度可视化和历史记录管理等功能。界面采用声明式开发范式，代码结构清晰，可维护性强。

（5）在华为 Mate 60 Pro 真机上进行了全面的系统测试。功能测试表明所有核心功能正常工作；性能测试显示单张图片推理耗时约 45ms，内存占用约 120MB，满足实时性要求；用户体验测试 SUS 评分为 82.5 分，表明应用具有良好的可用性。

6.2 存在的不足

尽管本文在基于鸿蒙系统与 MindSpore 的端侧 AI 应用开发方面取得了一定的成果，但仍存在以下不足：

（1）分类类别有限。本文仅使用 CIFAR-10 数据集进行训练和测试，仅支持 10 个类别的图像分类，实际应用场景中的类别数量远超于此。扩展到 ImageNet 的 1000 个类别或自定义类别时，模型精度和推理速度可能面临更大的挑战。

（2）模型结构单一。本文仅使用了 ResNet-50 一种网络结构，未对其他轻量化网络（如 EfficientNet、GhostNet 等）在鸿蒙端侧的推理性能进行对比分析。

（3）分布式能力利用不足。鸿蒙系统的核心优势之一是分布式能力，本文的应用仅在单设备上运行，未充分利用鸿蒙的分布式软总线、分布式数据管理等能力实现跨设备 AI 协同。

（4）动态更新能力缺失。当前应用的模型文件打包在应用内部，无法在不更新应用

的情况下更新模型。在实际应用中，模型可能需要定期更新以适应新的分类需求。

6.3 未来工作展望

针对上述不足，未来的研究工作可以从以下几个方面展开：

（1）扩展分类类别和模型能力。将分类类别从 10 类扩展到 100 类甚至 1000 类，引入目标检测、语义分割等更复杂的视觉任务，提升应用的实用性。同时，探索增量学习方法，使模型能够在端侧持续学习新的类别。

（2）探索更多轻量化模型。对比研究 EfficientNet、GhostNet、ShuffleNetV2 等轻量化网络在鸿蒙端侧的推理性能，寻找精度和效率的最优平衡点。研究神经架构搜索（NAS）技术在端侧模型设计中的应用。

（3）利用鸿蒙分布式能力。研究基于分布式软总线的跨设备模型推理方案，将计算密集型的推理任务卸载到附近的高性能设备上执行，实现设备间的 AI 能力共享。利用分布式数据管理实现模型和推理结果的跨设备同步。

（4）实现模型动态更新。设计基于鸿蒙应用市场的模型热更新机制，支持在不更新应用的情况下下载和替换模型文件，实现模型的持续迭代优化。

（5）增强隐私保护能力。研究联邦学习技术在端侧 AI 中的应用，在保护用户数据隐私的前提下实现模型的协同训练和优化，为端侧 AI 的隐私保护提供技术保障。

总之，随着鸿蒙系统生态的不断成熟和端侧 AI 技术的持续发展，基于鸿蒙系统的端侧 AI 应用将拥有广阔的应用前景。本文的研究工作为该领域的探索提供了有益的参考，期望未来能够看到更多高质量的端侧 AI 应用在鸿蒙生态中涌现。

致　谢

本论文是在导师李明老师的悉心指导下完成的。从论文选题、研究方案制定到论文撰写，李老师都给予了精心的指导和热情的帮助。李老师严谨的治学态度、渊博的专业知识和创新的研究思维，使我受益匪浅。在此，谨向李老师表示衷心的感谢和崇高的敬意！

感谢上海电机学院电气信息学院各位老师在本科学习期间对我的培养和教育，使我在软件工程专业领域打下了扎实的基础。特别感谢在鸿蒙开发和人工智能相关课程中给予我帮助的各位老师。

感谢华为开发者联盟和 MindSpore 开源社区提供的技术文档和开发工具，为本文的研究工作提供了重要支持。感谢开源社区中各位开发者的无私贡献。

感谢同实验室的各位同学：王磊、陈晓明、赵宇飞等，在项目开发过程中与我的讨论和协作，他们的建议对改进系统设计起到了重要作用。

最后，衷心感谢我的家人对我学业的支持和鼓励。他们的理解和关爱是我完成学业的强大动力。感谢父母多年来在生活上无微不至的关心和在精神上不断的鼓励，使我能够专心学业，顺利完成本科阶段的学习。

参考文献

[1] KRIZHEVSKY A, SUTSKEVER I, HINTON G E. ImageNet Classification with Deep Convolutional Neural Networks[C]//Advances in Neural Information Processing Systems. 2012: 1097-1105.

[2] ZHANG X, WANG Y, LU S. Edge Computing: Vision and Challenges[J]. IEEE Internet of Things Journal, 2020, 7(8): 7423-7437.

[3] LANE N D, BHATTACHARYA S, MATHUR P, et al. DeepX: A Software Accelerator for Low-Power Mobile Deep Learning Inference on Heterogeneous Processors [C]//ACM International Conference on Information Processing in Sensor Networks. 2016.

[4] Huawei Technologies. HarmonyOS Technical Documentation[Z]. https://developer.harmonyos.com. 2021.

[5] HAN S, MAO H, DALLY W J. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding[C]//International Conference on Learning Representations. 2016.

[6] HOWARD A G, ZHU M, CHEN B, et al. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications[C]//. 2017.

[7] WU C, LI W, WANG X, et al. Machine Learning Acceleration on Mobile Devices [J]. IEEE Transactions on Computers, 2019, 68(12): 1754-1767.

[8] MindSpore Community. MindSpore Lite Documentation[Z]. https://www.mindspore.cn/lite. 2021.

[9] HE K, ZHANG X, REN S, et al. Deep Residual Learning for Image Recognition[C]//IEEE Conference on Computer Vision and Pattern Recognition. 2016: 770-778.

[10] Huawei Technologies. ArkTS Language Reference[Z]. https://developer.harmonyos.com/cn/docs/documentation/doc-guides-V3/arkts-get-started-0000001504769028-V3. 2023.

[11] JACOB B, KLIGYS S, CHEN B, et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[C]//IEEE Conference on Computer Vision and Pattern Recognition Workshops. 2018.

[12] HINTON G, VINYALS O, DEAN J. Distilling the Knowledge in a Neural Network [J]. arXiv preprint arXiv:1503.02531, 2015.

附录 A　核心代码附录

A.1　ResNet-50 模型训练代码

以下为基于 MindSpore 框架的 ResNet-50 模型训练核心代码：

Listing A.1 ResNet-50 模型训练代码

```
import mindspore as ms
from mindspore import nn, context, Tensor
from mindspore.train import Model, LossMonitor
from mindspore.dataset import vision, transforms
from mindspore.dataset import Cifar10Dataset

context.set_context(mode=context.GRAPH_MODE, device_target="GPU")

# 数据预处理
transform_train = transforms.Compose([
    vision.RandomCrop(32, padding=4),
    vision.RandomHorizontalFlip(),
    vision.Resize(224),
    vision.Normalize(mean=[0.4914, 0.4822, 0.4465],
                     std=[0.2023, 0.1994, 0.2010]),
    vision.HWC2CHW()
])

# 数据集加载
dataset = Cifar10Dataset(dataset_dir="./cifar-10",
    usage="train", shuffle=True)
dataset = dataset.map(transform_train,
    input_columns="image")
dataset = dataset.batch(128)

# 模型定义
net = resnet50(num_classes=10)
loss_fn = nn.CrossEntropyLoss()
opt = nn.SGD(net.trainable_params(),
    learning_rate=0.1, momentum=0.9,
    weight_decay=5e-4)
model = Model(net, loss_fn, opt)

# 模型训练
model.train(200, dataset,
    callbacks=[LossMonitor(0.1)],
    dataset_sink_mode=True)
```

A.2　INT8 量化转换代码

以下为模型 INT8 后训练量化的核心代码：

Listing A.2 INT8 量化转换代码

```python
from mindspore_lite import Converter

converter = Converter()
converter.set_config_file("./quantization.cfg")
converter.set_model_file("resnet50.mindir")
converter.set_output_file("resnet50_quant.ms")
converter.set_weight_quant(True)
converter.set_input_shape("input:1,3,224,224")
converter.convert()
```

A.3 鸿蒙 NAPI 推理接口代码

以下为鸿蒙端侧推理引擎的 NAPI 接口核心代码：

Listing A.3 NAPI 推理接口代码

```cpp
#include <mindspore/lite/model.h>
#include <mindspore/lite/context.h>

napi_value Predict(napi_env env, napi_callback_info info) {
    // 获取输入数据
    size_t argc = 2;
    napi_value args[2];
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);

    // 创建推理上下文
    auto context = std::make_shared<mindspore::lite::Context>();
    context->device_list_ = {{mindspore::lite::DT_CPU, {}}};
    context->cpu_context_ = {
        .enable_float16_ = false,
        .thread_num_ = 4
    };

    // 创建模型并加载
    auto model = mindspore::lite::Model::CreateModel(
        model_path, context);

    // 创建推理器
    auto runner = mindspore::lite::ModelParallelRunner();
    runner.Init(model_path, context);

    // 执行推理
    auto inputs = runner.GetInputs();
    CopyInputData(inputs, input_buffer);
    auto outputs = runner.Predict(inputs);

    // 返回分类结果
    napi_value result;
    napi_create_string_utf8(env,
        GetTopResult(outputs).c_str(),
        NAPI_AUTO_LENGTH, &result);
    return result;
}
```

A.4 ArkTS 界面核心代码

以下为鸿蒙 ArkTS 应用主界面的核心代码：

Listing A.4 ArkTS 主界面代码

```
@Entry
@Component
struct IndexPage {
  @State imagePath: string = ''
  @State resultText: string = ''
  @State confidenceList: number[] = []

  build() {
    Column() {
      // 图片展示区域
      Image(this.imagePath)
        .width(300).height(300)
        .objectFit(ImageFit.Cover)
        .borderRadius(8)

      // 分类按钮
      Button('开始分类')
        .width('80%').height(40)
        .onClick(() => this.classifyImage())

      // 分类结果展示
      Text(this.resultText)
        .fontSize(20).fontWeight(FontWeight.Bold)

      // 置信度柱状图
      ForEach(this.confidenceList,
        (item: number, index: number) => {
          Row() {
            Progress({ value: item, total: 100 })
              .width('60%').color('#007DFF')
          }
        })
    }.width('100%').padding(20)
  }

  async classifyImage() {
    let result = await mindsporeLite.predict(
      this.imagePath)
    this.resultText = result.topClass
    this.confidenceList = result.confidences
  }
}
```